

Project Management System — Product Requirements Document

Version 1.1 · Implementation-Ready Specification · 8 October 2026 *Prepared for the ANTIGRAVITY AI coding agent and human reviewers (product, engineering, QA, security).*

1. Document Control

Field	Value
Product name	Project Management System (Responsive Web + Android)
Document title	Product Requirements Document (PRD)
Version	1.1 — Implementation-Ready Specification (v1.0 plus UI foundation §12.1–§12.6 and core-first slices §40.1)
Status	Final for implementation. Changes after approval are recorded in Open Decisions (§47).
Date	2026-10-08 (v1.1; v1.0 issued 2026-10-07)
Author / Owner	Assignment candidate (owner); drafted in the roles of Principal PM, Architect, Full-Stack Engineer, Security Engineer, QA Lead, UX Architect, Technical Writer
Source document	<i>Full Stack Developer Task: Project Management System (Web + Mobile) — Intern_Task_Full_Stack_Developer.pdf</i> , 7 pages. Authoritative source of truth.
Intended implementation platform	Web: React + Vite + TypeScript. Mobile: React Native + Expo (Android required, iOS optional). Backend: Node.js + Express + TypeScript (REST). Database: PostgreSQL via Prisma.
Intended audience	ANTIGRAVITY (primary implementer), evaluators, QA, security reviewers

1.1 Conventions and labels

Tag	Meaning
M	MANDATORY / ASSIGNMENT REQUIREMENT — stated in the PDF. Non-negotiable.
R	RECOMMENDED ENGINEERING DECISION — added here because the PDF is silent or leaves a choice open. Not mandated by the PDF.
B	OPTIONAL / BONUS — listed as optional in the PDF (bonus list, or iOS), or deliberately deferred. Never blocks mandatory work.

- Priority uses MoSCoW: **Must, Should, Could, Won't (this release)**. Every **M** item is Must.
- MUST / SHOULD / MAY follow RFC 2119.
- ID prefixes: AUTH- PROJ- TASK- DASH- SRCH- MOB- SYNC- SEC- API- NFR- DOC- DEP- (requirements); US- (user stories); AC- / TC- (acceptance criterion / test case — same suffix, so AC-AUTH-01 is verified by TC-AUTH-01); SRC- (PDF requirement, see §38); OD- (open decision, §47); ADR- (architecture decision, §42); SC- (scope item).
- Enum values on the wire are UPPER_SNAKE_CASE (NOT_STARTED , IN_PROGRESS , COMPLETED , PENDING , LOW , MEDIUM , HIGH). UI labels use the PDF wording (Not Started, In Progress, Completed, Pending, Low, Medium, High).
- PDF field and metric names are preserved as UI labels: Full Name, Email Address, Password, Project Name, Description, Status, Start Date, End Date, Created Date, Task Name, Priority, Due Date, Total Projects, Total Tasks, Completed Tasks, Pending Tasks, Projects In Progress.

2. Executive Summary

What it is. A Project Management System that lets an individual user create projects, organize tasks inside them, track progress, and see overall statistics on a dashboard. It ships as a responsive web application and an Android mobile application.

Why it exists. The source assignment evaluates whether one engineer can deliver a secure, well-structured, cross-platform product on a single shared backend. A user must be able to register or log in on either platform and see and manage the same projects and tasks.

Who uses it. Authenticated individuals managing their own projects and tasks.

Unauthenticated visitors can only reach Login and Register.

Main capabilities. Registration, login, logout; project CRUD; task CRUD with completion, status and priority changes; a five-metric dashboard; server-side search and filtering; pull-to-refresh on mobile; secure token storage; token-expiry and no-network handling.

Architecture. Three clients-of-one-API: React web SPA and React Native (Expo) Android app both call one Node.js/Express REST API (modular monolith), which uses one PostgreSQL database. There is no mobile-specific backend and no independent mobile database for primary data. The backend and database are the single source of truth.

Security philosophy. Deny by default. Every protected endpoint requires a valid JWT; every query is scoped to the authenticated owner; all input is validated on the backend regardless of which client sent it; passwords are bcrypt-hashed; queries go through the ORM; authentication endpoints are rate limited; secrets live only in environment variables.

Expected final deliverable. A public Git repository (backend, web, mobile, docs), a deployed backend + PostgreSQL, a deployed web app, an installable Android build (APK or Expo/Firebase distribution), database schema/ER diagram, API documentation, README, and a 5-minute screen recording demonstrating the same account on web and mobile with a task created on one platform and shown on the other.

3. Product Vision

One unified project and task management experience across web and Android, backed by one account, one API and one database. Users trust that what they see on a phone matches what they see in a browser (after a refresh), and that nobody else can see or change their data. The product should feel like a polished, professionally engineered tool — clear, consistent, accessible and fast — not a minimal CRUD demo, while staying simple enough that its author can explain every design decision in a review.

4. Goals

ID	Goal	Measure of success
G-01	Secure user authentication	Register/login/logout/me work; bcrypt + JWT; auth rate limiting verified (AC-AUTH-01..09)
G-02	Project lifecycle management	Create/view/list/edit/delete with ownership enforcement (AC-PROJ-01..09)

ID	Goal	Measure of success
G-03	Task lifecycle management	Create/view/edit/delete/complete/status/priority (AC-TASK-01..10)
G-04	Dashboard visibility	Five required metrics, scoped to the authenticated user (AC-DASH-01..04)
G-05	Search and filtering	Server-side search + filters for projects and tasks (AC-SRCH-01..06)
G-06	Cross-platform consistency	Web↔mobile changes visible after refresh (AC-SYNC-01..03)
G-07	Secure API architecture	Zero cross-user access in the authorization matrix (AC-SEC-03)
G-08	Responsive web experience	Usable from 320 px to 1920 px wide without horizontal page scroll
G-09	Usable Android experience	All mandatory mobile flows completable one-handed on a phone viewport (AC-MOB-01..10)
G-10	Maintainable architecture	Layered backend, componentized clients, strict TypeScript, no business logic in routes
G-11	Deployable application	Backend, DB, web and Android build reachable and working (AC-DEP-01..04)
G-12	Complete documentation	A new developer runs all three apps from the README alone (AC-DOC-01..03)

5. Non-Goals

Outside mandatory scope (RECOMMENDED ENGINEERING DECISION to keep them out; none is required by the PDF):

- Team collaboration, project sharing, task assignment to other users, invitations.
- Real-time collaboration, WebSockets, live push of changes between devices (refresh is sufficient per PDF).
- Complex role hierarchy (Role-Based Access Control is an OPTIONAL/BONUS item only).
- Billing, subscriptions, chat, comments, file attachments, notifications (push notifications are BONUS).
- External integrations (calendar, Slack, email). Password reset and email verification flows are not in the PDF and are out of scope.

- Microservices, message queues, caching layers, GraphQL.
- Server-side session revocation (see OD-03).
- iOS build (OPTIONAL per PDF; the code SHOULD remain iOS-compatible but iOS is not delivered or verified).

6. Users / Personas

Persona	Description	Can	Cannot
Primary User — authenticated individual project/task manager	Registers once; uses the same account on web and Android	Manage own projects and tasks, see own dashboard, search/filter own data, log out	See, modify or delete any other user's data; see other users' existence or counts
Unauthenticated Visitor	Has no valid token	Open Login and Register screens; submit register/login requests	Access any project, task, dashboard or profile data; any protected route redirects to Login
Evaluator / Reviewer (non-product persona)	Reads repo, runs apps, watches the demo	Use a seeded demo account with test data only	—

Note: all data in seeds, tests and the demo MUST be synthetic test data. No real personal data (PDF Important Notes).

7. Scope

ID	Feature	Priority	Required/Bonus	Description
SC-01	Authentication	Must	M	Register, login, logout, current user, JWT session, one account on web + mobile
SC-02	Project management	Must	M	Create, view, list (owned), edit, delete projects with the six PDF fields

ID	Feature	Priority	Required/Bonus	Description
SC-03	Task management	Must	M	Create, edit, delete, complete, view tasks under a project; seven PDF fields incl. priority/status
SC-04	Dashboard	Must	M	Total Projects, Total Tasks, Completed Tasks, Pending Tasks, Projects In Progress
SC-05	Project search and filter	Must	M	Search by name; filter by status
SC-06	Task search and filter	Must	M	Search by name; filter by status; filter by priority
SC-07	Web application	Must	M	React, responsive, component structure, form validation, loading indicators, error handling
SC-08	Android application	Must	M	Auth, dashboard, view projects + tasks, task CRUD, complete/status/priority, search/filter, pull-to-refresh
SC-09	Mobile project create/edit/delete	Should	R	Not required by PDF; recommended so a user who registers on mobile is not blocked from creating tasks
SC-10	iOS build	Could	B	Optional per PDF
SC-11	Cross-platform synchronization	Must	M	Shared backend/DB; changes visible on the other platform after refresh
SC-12	Mobile session robustness	Must	M	Secure token storage, expired-token → login with message, no-network message
SC-13	Backend engineering	Must	M	REST, route organization, middleware, error handling, logging, clean structure, CORS for web domain

ID	Feature	Priority	Required/Bonus	Description
SC-14	Relational database	Must	M	PostgreSQL, foreign keys, normalized schema
SC-15	Security	Must	M	bcrypt, JWT, auth middleware, authorization, validation, SQL-injection protection, auth rate limiting
SC-16	REST API (15 endpoints)	Must	M	Exact minimum endpoint set used by both clients
SC-17	Documentation	Must	M	Setup (backend/web/mobile), env vars, DB setup, API docs, mobile against deployed backend
SC-18	Deployment	Must	M	Deployed web + backend; Android APK or Expo/Firebase distribution
SC-19	Security-verification tests	Should	R	Cross-user, validation, rate-limit tests that prove mandatory security (P0 verification)
SC-20	Submission package	Must	M	Public repo, schema/ER, API docs, README, URLs, build, 5-minute recording
SC-21	Bonus capabilities	Could	B	Docker, broader unit/integration tests, pagination, sorting, audit logs, RBAC, CI/CD, refresh tokens, due-tomorrow push, offline viewing, shared types

8. Functional Requirements

Each feature has a card (ID, objective, actors, preconditions, priority, API dependencies), then individually testable requirements. Acceptance criteria are defined in §27 (AC-xxx), each verified by a same-suffix test case (TC-xxx, §26). API contracts, the response envelope and error codes are in §18–§19. Edge cases are consolidated in §28.

8A. Authentication (F-AUTH)

Item	Detail
Feature ID / priority	F-AUTH · Must · M
Objective	Let a person create an account and securely access only their own workspace from web or Android with the same credentials
Actors	Unauthenticated Visitor (register/login); Primary User (me/logout)
Preconditions	Backend reachable; database migrated; <code>JWT_SECRET</code> configured (≥ 32 random bytes)
API dependencies	POST <code>/api/auth/register</code> , POST <code>/api/auth/login</code> , POST <code>/api/auth/logout</code> , GET <code>/api/auth/me</code>
Acceptance	AC-AUTH-01..09

User flow — register: open Register → enter Full Name, Email Address, Password → client validates → submit → server validates, hashes, stores → returns user + token → client stores token securely → Dashboard. **User flow — login:** open Login → Email Address + Password → submit → token stored → Dashboard. **Logout:** profile/menu → Log out → client clears token and cached data → Login screen.

ID	Requirement	Tag
AUTH-001	Registration accepts exactly three user fields: <code>fullName</code> (Full Name), <code>email</code> (Email Address), <code>password</code> (Password). All three are required.	M
AUTH-002	<code>fullName</code> : trimmed, 1-100 chars, not blank. Limits are recommended values (OD-13).	R
AUTH-003	<code>email</code> : trimmed, lowercased before use/storage, valid email format, ≤ 254 chars. Invalid format → 400 with field detail.	M (validation) / R (limits)
AUTH-004	Email uniqueness enforced twice: application check and a database UNIQUE constraint. A race that hits the constraint MUST map to 409, not 500.	M

ID	Requirement	Tag
AUTH-005	password policy: 8–72 characters (bcrypt input limit), at least one letter and one digit, not blank/whitespace-only, not trimmed or altered. Empty password → 400.	R
AUTH-006	Passwords are hashed with bcrypt (cost factor ≥ 12; bcryptjs acceptable). Only passwordHash is stored. Plain-text passwords are never stored, logged, or returned.	M
AUTH-007	Duplicate registration returns 409 EMAIL_ALREADY_REGISTERED ; no second row is created.	M
AUTH-008	Successful registration returns 201 with { data: { user, token, expiresAt } } (user auto-signed-in). user = id, fullName, email, createdAt only (OD-09).	R
AUTH-009	Login accepts email + password . Email is normalized as in AUTH-003. Any failure (unknown email, wrong password) returns the same 401 INVALID_CREDENTIALS body and message Invalid email or password.	M / R
AUTH-010	For unknown emails the server MUST still run a bcrypt comparison against a dummy hash to equalize response timing.	R
AUTH-011	Login success returns 200 { data: { user, token, expiresAt } } . Token is a signed JWT (HS256) with claims sub (user id), iat , exp ; no email, name or other personal data in the payload.	M (JWT) / R (claims)
AUTH-012	Token lifetime comes from JWT_EXPIRES_IN (default 1d , OD-05). Expired tokens are rejected.	M / R
AUTH-013	Rate limiting on authentication endpoints: login — 10 failed attempts per IP per 15 minutes (successful logins not counted); register — 10 requests per IP per hour. Exceeded → 429 RATE_LIMITED with Retry-After header. Limits are env-configurable. trust proxy MUST be set correctly in deployment so the client IP is real.	M (existence) / R (values)

ID	Requirement	Tag
AUTH-014	POST /api/auth/logout is stateless and idempotent: always returns 204 (valid, expired, or missing token). Clients clear the token and cached data and navigate to Login. No server-side revocation (OD-03).	M (endpoint) / R (semantics)
AUTH-015	GET /api/auth/me requires a valid token; returns { data: { user } } with id, fullName, email, createdAt. MUST NOT return passwordHash or any internal field.	M
AUTH-016	Session persistence: the user stays logged in until logout or token expiration. On app start each client reads the stored token and validates it with GET /api/auth/me before showing protected screens.	M
AUTH-017	Missing/invalid/expired token on a protected route → 401 with distinct codes UNAUTHENTICATED, TOKEN_INVALID, TOKEN_EXPIRED. The client clears the session and shows Login with Your session has expired. Please log in again. (for TOKEN_EXPIRED) or a neutral sign-in prompt (others).	M
AUTH-018	A valid token whose user no longer exists is treated as TOKEN_INVALID (401).	R
AUTH-019	Authentication middleware protects every route except register, login, logout, health check and API docs. New routes are protected by default (deny-by-default router mounting).	M / R
AUTH-020	One account works on both platforms: no platform-specific account data; web and mobile call identical endpoints with identical payloads.	M
AUTH-021	Accepted trade-off: registration reveals whether an email exists (409). Mitigated by rate limiting. Login never reveals it.	R

UI requirements. Login and Register screens on web and mobile: labeled inputs, inline field errors on blur/submit, password show/hide toggle, submit button disabled with spinner while pending (prevents duplicate submits), server errors in an accessible alert

region, link between the two screens, `autocomplete` attributes (`name` , `email` , `current-password` / `new-password`) on web, correct keyboard types on mobile. **Security considerations.** Never log request bodies for auth routes; redact `password` and `Authorization` in logs; no tokens in URLs; generic credential errors; HTTPS only in production.

8B. Project Management (F-PROJ)

Item	Detail
Feature ID / priority	F-PROJ · Must · M (web full CRUD; mobile view-only is Must, mobile create/edit/delete is Should/R)
Objective	Let a user organize work into projects with status and dates and see progress
Actors	Primary User
Preconditions	Authenticated; token valid
API dependencies	GET/POST <code>/api/projects</code> ; GET/PUT/DELETE <code>/api/projects/{id}</code>
Acceptance	AC-PROJ-01..09

Project fields (M): Project Name, Description, Status (Not Started, In Progress, Completed), Start Date, End Date, Created Date, plus `id` and owner relationship.

ID	Requirement	Tag
PROJ-001	Create project accepts <code>name</code> (required), <code>description</code> , <code>status</code> , <code>startDate</code> , <code>endDate</code> . <code>name</code> : trimmed, 1-120 chars. <code>description</code> : optional, ≤ 2000 chars, default empty string. <code>status</code> : default <code>NOT_STARTED</code> .	M (fields) / R (limits, defaults)
PROJ-002	The server sets <code>id</code> (UUID), <code>createdAt</code> (UTC, Created Date), <code>updatedAt</code> and <code>ownerId</code> (from the JWT). Client-supplied <code>id</code> , <code>ownerId</code> , <code>createdAt</code> , <code>updatedAt</code> are rejected with 400 (strict schema; no mass assignment).	M / R

ID	Requirement	Tag
PROJ-003	<code>status</code> MUST be one of <code>NOT_STARTED</code> , <code>IN_PROGRESS</code> , <code>COMPLETED</code> ; any other value → 400.	M
PROJ-004	<code>startDate</code> / <code>endDate</code> are calendar dates <code>YYYY-MM-DD</code> and MUST be real dates (2026-02-30 → 400). Both optional and nullable (OD-01); <code>null</code> clears a date. When both are present, <code>endDate</code> ≥ <code>startDate</code> (equal allowed). On update the rule is applied to the merged (stored + supplied) values.	M (valid dates) / R (optional, ordering)
PROJ-005	Empty-string rules: <code>name</code> empty or whitespace → 400; <code>description</code> may be empty; a date sent as <code>""</code> → 400 (use <code>null</code> to clear).	M
PROJ-006	GET <code>/api/projects</code> returns only projects owned by the caller, default order <code>createdAt</code> descending, supporting <code>search</code> and <code>status</code> (§30).	M
PROJ-007	Project responses include <code>taskCount</code> and <code>completedTaskCount</code> (computed with one aggregate query, no N+1) so clients can show progress.	R
PROJ-008	GET <code>/api/projects/{id}</code> returns the project (same shape as list items) if owned by the caller.	M
PROJ-009	PUT <code>/api/projects/{id}</code> updates any subset of <code>name</code> , <code>description</code> , <code>status</code> , <code>startDate</code> , <code>endDate</code> (at least one field required) — partial-update semantics (OD-07).	M (endpoint) / R (semantics)
PROJ-010	DELETE <code>/api/projects/{id}</code> returns 204 and deletes the project and all its tasks atomically (FK <code>ON DELETE CASCADE</code>). The UI MUST show a confirmation naming the project and its task count first (OD-02, §29).	M (delete) / R (cascade)

ID	Requirement	Tag
PROJ-011	Ownership: every read/update/delete query includes <code>id = :id AND ownerId = :callerId</code> . A non-owned or non-existent id returns the same 404 <code>PROJECT_NOT_FOUND</code> (no existence leak, OD-11).	M
PROJ-012	A malformed <code>id</code> path parameter (not a UUID) $\rightarrow$ 400 <code>VALIDATION_ERROR</code> .	R
PROJ-013	Project <code>status</code> is manual. Completing all tasks does NOT change project status automatically.	R
PROJ-014	Web screens: Project List, Project Details (with its tasks), Create Project, Edit Project; each with loading, empty, error (with Retry) and success feedback.	M
PROJ-015	Mobile: Projects list and Project details with tasks are required (Must). Create/edit/delete project on mobile is Should (R) and ships only after all mandatory mobile items pass.	M / R
PROJ-016	Status is shown as a badge with text plus color (never color alone). Project list item shows name, status badge, date range, and <code>completedTaskCount / taskCount</code> .	R

UX copy. Empty: You don't have any projects yet. (+ primary button *Create project* on web). Delete dialog: Delete “{name}”? This will also permanently delete its {n} task(s). This action cannot be undone. Success toasts: Project created. / Project updated. / Project deleted. **Error cases.** 400 invalid body; 401 session; 404 not found/not owned; 429 not applicable; 500 generic Something went wrong. Please try again. **Security considerations.** IDOR/BOLA: ownership predicate in the data-access layer, not only in controllers; never trust `ownerId` from clients; list endpoints never return rows without an owner predicate. **Edge cases.** See §28 (empty name, invalid status, end before start, delete with tasks, very long names).

8C. Task Management (F-TASK)

Item	Detail
Feature ID / priority	F-TASK · Must · M
Objective	Let a user break a project into tasks, track priority and status, and complete them
Actors	Primary User (web and Android)
Preconditions	Authenticated; the target project exists and is owned by the caller
API dependencies	GET/POST /api/tasks; GET/PUT/DELETE /api/tasks/{id}
Acceptance	AC-TASK-01..10

Task fields (M): Task Name, Description, Priority (Low, Medium, High), Status (Pending, In Progress, Completed), Due Date, Created Date, plus `id` and `projectId` (each project can contain multiple tasks).

ID	Requirement	Tag
TASK-001	Create task accepts <code>projectId</code> (required UUID), <code>name</code> (required, trimmed, 1-150 chars), <code>description</code> (optional, ≤ 2000, default empty), <code>priority</code> (optional, default <code>MEDIUM</code>), <code>status</code> (optional, default <code>PENDING</code>), <code>dueDate</code> (optional <code>YYYY-MM-DD</code> , nullable).	M (fields) / R (limits, defaults)
TASK-002	Server sets <code>id</code> , <code>createdAt</code> (Created Date), <code>updatedAt</code> . Client-supplied <code>id</code> , <code>createdAt</code> , <code>updatedAt</code> or any user field → 400 (strict schema).	M / R
TASK-003	<code>projectId</code> MUST reference a project owned by the caller. Non-owned and non-existent projects both return 404 <code>PROJECT_NOT_FOUND</code> and create nothing.	M
TASK-004	<code>priority</code> ∈ <code>LOW</code> , <code>MEDIUM</code> , <code>HIGH</code> ; <code>status</code> ∈ <code>PENDING</code> , <code>IN_PROGRESS</code> , <code>COMPLETED</code> . Anything else → 400.	M

ID	Requirement	Tag
TASK-005	<code>dueDate</code> must be a real calendar date. Past dates are allowed (OD-15). <code>""</code> → 400; <code>null</code> clears.	M (valid date) / R
TASK-006	<code>name</code> empty or whitespace → 400; <code>description</code> may be empty.	M
TASK-007	Ownership is resolved through the task's project: every task query joins <code>project.ownerId = :callerId</code> . The owner is never taken from client input.	M
TASK-008	GET <code>/api/tasks</code> returns the caller's tasks across all owned projects. Query: <code>projectId</code> , <code>search</code> , <code>status</code> , <code>priority</code> (§30). Default order <code>createdAt</code> descending. Each item includes <code>projectId</code> and <code>project: { id, name }</code> .	M / R
TASK-009	View tasks under a project = GET <code>/api/tasks?projectId={id}</code> . A <code>projectId</code> the caller does not own → 404 <code>PROJECT_NOT_FOUND</code> (not an empty list).	M / R
TASK-010	GET <code>/api/tasks/{id}</code> returns the task if its project is owned by the caller, else 404 <code>TASK_NOT_FOUND</code> .	M
TASK-011	PUT <code>/api/tasks/{id}</code> updates any subset of <code>name</code> , <code>description</code> , <code>priority</code> , <code>status</code> , <code>dueDate</code> (≥ 1 field). <code>projectId</code> is immutable in v1 — supplying it → 400 (OD-07, OD-08).	M (endpoint) / R
TASK-012	Mark completed = PUT <code>{ status: COMPLETED }</code> . Idempotent: completing an already-completed task returns 200 with the task unchanged. Reverting to <code>PENDING / IN_PROGRESS</code> is allowed.	M / R
TASK-013	Quick actions without opening the full edit form: complete toggle, change status, change priority (inline control/menu on web, action sheet on mobile).	M
TASK-014	DELETE <code>/api/tasks/{id}</code> → 204. UI confirmation first: Delete this task? This action cannot be undone. Non-owned/non-existent → 404.	M / R

ID	Requirement	Tag
TASK-015	Completing tasks never changes project status automatically.	R
TASK-016	Visual semantics: completed = check icon + strikethrough + text label; priority = icon + text + color; Overdue label when dueDate < today (client local date) and status ≠ COMPLETED . Never color alone.	R
TASK-017	Search/filters combine with AND and, inside Project Details, are scoped to that projectId .	M / R
TASK-018	Concurrency: last write wins (no optimistic locking). Clients refetch after each mutation.	R
TASK-019	States. Loading: skeleton rows/spinner. Empty (project): This project has no tasks yet. ; empty (global): You don't have any tasks yet. ; empty (filtered): No tasks match your search or filters. Error: message + Retry. Success: Task created. / Task updated. / Task deleted.	M / R
TASK-020	Web screens: Project Details task section (list, add, edit, quick actions) and a global Tasks page. Mobile: Tasks tab, Project Details tasks, Task Form (create/edit), Task Details with quick actions.	M

Security considerations. BOLA/IDOR on both **/tasks/{id}** and **projectId** in bodies and queries; mass-assignment blocked by strict schemas; validation identical regardless of client. **Edge cases.** §28 (empty name, invalid priority/status/due date, foreign project, completing a completed task, long text, double submit).

8D. Dashboard (F-DASH)

Item	Detail
Feature ID / priority	F-DASH · Must · M
Objective	Give the user an at-a-glance view of their own workload
Actors	Primary User

Item	Detail
Preconditions	Authenticated
API dependencies	GET /api/dashboard
Acceptance	AC-DASH-01..04

ID	Requirement	Tag
DASH-001	GET /api/dashboard requires authentication and returns the five required metrics: <code>totalProjects</code> , <code>totalTasks</code> , <code>completedTasks</code> , <code>pendingTasks</code> , <code>projectsInProgress</code> (UI labels: Total Projects, Total Tasks, Completed Tasks, Pending Tasks, Projects In Progress).	M
DASH-002	Definitions. <code>totalProjects</code> = projects owned by caller. <code>totalTasks</code> = tasks in caller's projects. <code>completedTasks</code> = those with status <code>COMPLETED</code> . <code>pendingTasks</code> = those with status <code>PENDING</code> (OD-04). <code>projectsInProgress</code> = caller's projects with status <code>IN_PROGRESS</code> .	M / R
DASH-003	Extra field <code>inProgressTasks</code> (tasks with status <code>IN_PROGRESS</code>) is returned so that <code>totalTasks</code> = <code>pendingTasks</code> + <code>inProgressTasks</code> + <code>completedTasks</code> always reconciles.	R
DASH-004	Scope: derived solely from the JWT user. No parameter can select another user. Another user's rows are never counted.	M
DASH-005	Computed in the database with <code>COUNT/GROUP BY</code> (no loading of all rows); ≤ 3 queries, preferably one transaction.	R
DASH-006	Always fresh: computed per request, <code>Cache-Control: no-store</code> . Web refetches on entry, window focus, and after any project/task mutation. Mobile refetches on screen focus and pull-to-refresh.	M (updates with user data) / R

ID	Requirement	Tag
DASH-007	UI: five stat cards in the PDF order. Web grid: 1 column < 640 px, 2 columns ≥ 640 px, 5 across ≥ 1280 px. Mobile: two-column card grid. Each card = number + label + icon.	R
DASH-008	Loading: skeleton cards. Empty (all zeros): cards show 0 and a hint <code>Nothing here yet. Create your first project to get started.</code> Error: inline alert <code>Unable to load your dashboard.</code> + Retry. 401 → session-expired flow.	R

8E. Search and Filtering (F-SRCH)

Item	Detail
Feature ID / priority	F-SRCH · Must · M
Objective	Find projects and tasks quickly, using server-side querying
API dependencies	GET /api/projects (<code>search</code> , <code>status</code>); GET /api/tasks (<code>search</code> , <code>status</code> , <code>priority</code> , <code>projectId</code>)
Acceptance	AC-SRCH-01..06

ID	Requirement	Tag
SRCH-001	Projects: search by project name; filter by status.	M
SRCH-002	Tasks: search by task name; filter by status; filter by priority.	M
SRCH-003	Matching is case-insensitive and partial (substring anywhere in the name).	R
SRCH-004	<code>search</code> is trimmed; empty after trimming is ignored; max 100 chars (else 400). <code>%</code> , <code>_</code> and <code>\</code> in the term are matched literally.	R
SRCH-005	All supplied parameters combine with AND; omitted parameters impose no restriction. Example: <code>GET /api/tasks?search=api&status=IN_PROGRESS&priority=HIGH.</code>	R

ID	Requirement	Tag
SRCH-006	Invalid enum values or repeated/array parameters → 400 <code>VALIDATION_ERROR</code> . Unknown query parameters are ignored (reserved for bonus <code>page</code> , <code>limit</code> , <code>sort</code>).	M (validation) / R
SRCH-007	Filtering is authoritative on the server; clients MUST send parameters to the API and MUST NOT rely on client-side filtering.	R
SRCH-008	Safe querying: ORM parameterized queries only; no raw SQL containing user input.	M
SRCH-009	Client UX: debounce typing 300 ms; cancel stale in-flight requests; keep previous results visible (dimmed) with an inline loading indicator; status/priority controls include an <code>All</code> option (parameter omitted); a <code>Clear filters</code> control appears when any search/filter is active and restores the unfiltered list. Web keeps search/filter state in the URL query so reload and Back work.	R
SRCH-010	Empty result: <code>No projects match your search or filters.</code> / <code>No tasks match your search or filters.</code> plus <code>Clear filters</code> — distinct from the empty-account state.	R

8F. Mobile Application (F-MOB)

Item	Detail
Feature ID / priority	F-MOB · Must · M (Android); iOS Could/B
Objective	A phone-first client of the same API with the same account
Preconditions	Android 8.0 (API 26)+ device or emulator, network reachable to the deployed backend (R)
API dependencies	All 15 endpoints (mobile uses the subset it needs, with identical contracts)
Acceptance	AC-MOB-01..10, AC-SYNC-01..03

Screens.

Screen	Purpose and key elements	States
Splash / Loading	Read token from secure storage; call GET /api/auth/me; route to Dashboard or Login	spinner; network error → retry; invalid token → Login
Login	Email Address, Password, Log in, link to Register	validating, submitting, error
Register	Full Name, Email Address, Password, Create account	validating, submitting, 409/400 errors
Dashboard	Five stat cards; pull-to-refresh	loading, empty, error+retry
Projects	List with name, status badge, progress; search by name + status filter (Should, R); pull-to-refresh	loading, empty, filtered-empty, error
Project Details	Project info; its tasks with search/filter; Add task button	loading, empty (This project has no tasks yet.), error
Tasks	All tasks; search; status and priority filters; quick complete	loading, empty, filtered-empty, error
Task Form	Create/edit: Task Name, Description, Priority, Status, Due Date (date picker), project selector on create	validating, submitting, error
Task Details	Full task; quick actions (complete, status, priority), Edit, Delete (confirm)	loading, error, not-found
Profile	Full Name, Email Address, Log out	—

ID	Requirement	Tag
MOB-001	Android is delivered and verified (installable APK or distribution link). iOS is OPTIONAL; code should avoid Android-only assumptions but iOS is neither delivered nor tested.	M / B
MOB-002	The app uses the same backend and database through the same endpoints. No second backend; no local database for primary data.	M

ID	Requirement	Tag
MOB-003	Register, log in, log out with the same account used on web.	M
MOB-004	View the dashboard (five metrics).	M
MOB-005	View all own projects and the tasks under each project.	M
MOB-006	Create, edit and delete tasks (delete with confirmation).	M
MOB-007	Mark tasks completed; change task status; change task priority.	M
MOB-008	Search tasks by name; filter tasks by status and by priority (server-side).	M
MOB-009	Pull-to-refresh on Dashboard, Projects, Project Details and Tasks fetches fresh data from the server, shows the native refresh indicator, and on failure keeps prior data and shows a non-blocking error.	M
MOB-010	Token stored only in secure device storage (Expo SecureStore → Android Keystore-backed; iOS Keychain). Never in AsyncStorage/MMKV/plain files, logs or the JS bundle. Deleted on logout and on session expiry.	M
MOB-011	On 401 <code>TOKEN_EXPIRED</code> / <code>TOKEN_INVALID</code> the shared API client clears secure storage and query cache, resets navigation to Login (handled once even if several requests fail), and shows <code>Your session has expired. Please log in again.</code>	M
MOB-012	No network or timeout (15 s) shows <code>Unable to connect. Please check your internet connection and try again.</code> with Retry. No crash, no blank screen. Forms keep their input on failure.	M / R
MOB-013	Form validation on Login, Register and Task Form with inline messages; rules mirror the server; the server remains authoritative.	M

ID	Requirement	Tag
MOB-014	Navigation: auth stack (Login, Register) and app tabs (Dashboard, Projects, Tasks, Profile) with stack screens for details and forms. Android hardware Back works.	M / R
MOB-015	Phone-first design: single column, ≥ 48 dp touch targets, safe areas, keyboard avoidance, floating Add button, native pickers/action sheets. Not a shrunken desktop UI.	M / R
MOB-016	Mobile project create/edit/delete (Should, R): implement only after MOB-001..015 pass.	R
MOB-017	Production builds use HTTPS only (no cleartext traffic); cleartext allowed only in local development builds. Permissions limited to what is needed (network).	R
MOB-018	API base URL is configured via <code>EXPO_PUBLIC_API_BASE_URL</code> (documented as <code>API_BASE_URL</code> in §32; the prefix is required by Expo to inline the value).	R
MOB-019	Accessibility: labels on icon buttons, sufficient contrast, support for system font scaling.	R

8G. Cross-Platform Synchronization (F-SYNC)

ID	Requirement	Tag
SYNC-001	The backend and PostgreSQL are the single source of truth. No client holds authoritative data.	M
SYNC-002	Both clients use identical endpoints, payloads and enums. A change on one platform is visible on the other after a refresh (web: reload, refresh button, or refetch on focus; mobile: pull-to-refresh). Real-time push is not required.	M
SYNC-003	Client caches (TanStack Query) MUST be invalidated after the client's own mutations and bypassed on explicit refresh. Recommended <code>staleTime</code> ≤ 30 s.	R

ID	Requirement	Tag
SYNC-004	The same account has the same <code>user.id</code> on both platforms.	M
SYNC-005	Date-only fields (start, end, due) carry no time zone and are displayed unchanged on both platforms; <code>createdAt</code> is UTC ISO-8601 shown in local time.	R
SYNC-006	Conflict policy: last write wins; documented in README.	R

Scenario	Steps	Expected result
S1 Web → Mobile create	Create task on web; open mobile; pull-to-refresh	Task appears with identical fields
S2 Mobile → Web update	Change task status on mobile; refresh web	Updated status shown on web; dashboard counts updated
S3 Web → Mobile delete	Delete task on web; pull-to-refresh on mobile	Task no longer listed
S4 Mobile → Web create (R)	Create task on mobile; refresh web	Task appears

```
sequenceDiagram
    participant W as Web app
    participant A as Express API
    participant D as PostgreSQL
    participant M as Android app
    W->>A: POST /api/tasks (Bearer token)
    A->>D: INSERT task (project owned by caller)
    D-->>A: new row
    A-->>W: 201 task
    M->>A: GET /api/tasks (pull-to-refresh)
    A->>D: SELECT tasks of caller
    D-->>A: rows incl. new task
    A-->>M: 200 task list
```

9. User Stories

Format: As a *[user]*, I want to *[action]*, so that *[benefit]*. Acceptance criteria are summarized per story and map to the AC matrix (§27).

ID	Story	Acceptance criteria
US-AUTH-001	As a new user, I want to register with my Full Name, Email Address and Password, so that I can access my project workspace.	Valid registration → 201 + session; duplicate email → 409; invalid email/missing field/weak password → 400; password never stored in plaintext (AC-AUTH-01..03)
US-AUTH-002	As a returning user, I want to log in with my email and password, so that I can continue my work.	Valid login → 200 + JWT; wrong credentials → generic 401; 11th failed attempt in 15 min → 429 (AC-AUTH-04, 05)
US-AUTH-003	As a logged-in user, I want to log out, so that nobody else can use my session on this device.	Token and cached data cleared; Login screen shown; logout is idempotent (AC-AUTH-08)
US-AUTH-004	As a user, I want to stay logged in until I log out or my token expires, so that I don't re-enter credentials constantly.	App restart keeps session while token valid; <code>/me</code> revalidates (AC-AUTH-06)
US-AUTH-005	As a user whose session expired, I want a clear message and a return to Login, so that I understand what happened.	401 <code>TOKEN_EXPIRED</code> → session cleared → Login + <code>Your session has expired. Please log in again.</code> (AC-AUTH-07, AC-MOB-09)
US-AUTH-006	As a user, I want one account to work on web and Android, so that I can switch devices freely.	Register on one, log in on the other; same <code>user.id</code> (AC-AUTH-09)
US-PROJ-001	As a user, I want to create a project with name, description, status and dates, so that I can organize my work.	201 with owner = me, default status Not Started; empty name/invalid status/invalid dates/end before start → 400 (AC-PROJ-01..03)
US-PROJ-002	As a user, I want to see all my projects, so that I can choose what to work on.	Only my projects listed; loading, empty (<code>You don't have any projects yet.</code>) and error states (AC-PROJ-04, 08)
US-PROJ-003	As a user, I want to open a project and see its details and tasks, so that I can track progress.	Fields shown; task counts shown; foreign id → 404 (AC-PROJ-05, AC-TASK-09)

ID	Story	Acceptance criteria
US-PROJ-004	As a user, I want to edit a project, so that I can keep it up to date.	Partial update persists; unknown/forbidden fields → 400; foreign id → 404 (AC-PROJ-05, 06)
US-PROJ-005	As a user, I want to delete a project after a clear warning, so that I can remove finished or mistaken work.	Confirmation shows task count; deletion cascades; later GET → 404 (AC-PROJ-07, 08)
US-PROJ-006	As a user, I want to search projects by name and filter by status, so that I can find a project fast.	Case-insensitive partial name match; status filter; combinable; Clear filters (AC-SRCH-01, 02, 06)
US-TASK-001	As a user, I want to create a task in a project with name, description, priority, status and due date, so that I can plan work.	201 with defaults Medium/Pending; foreign project → 404; invalid fields → 400 (AC-TASK-01..03)
US-TASK-002	As a user, I want to see the tasks under a project, so that I know what remains.	Only that project's tasks; empty state This project has no tasks yet. (AC-TASK-09)
US-TASK-003	As a user, I want to edit a task, so that details stay accurate.	Partial update persists; projectId change rejected; foreign task → 404 (AC-TASK-05)
US-TASK-004	As a user, I want to delete a task after confirmation, so that I can drop irrelevant work.	204; confirmation shown; foreign task → 404 (AC-TASK-06, 10)
US-TASK-005	As a user, I want to mark a task completed in one action, so that progress is recorded quickly.	Status becomes Completed; repeating is harmless; dashboard updates (AC-TASK-07, AC-DASH-04)
US-TASK-006	As a user, I want to change a task's status, so that its state reflects reality.	Pending / In Progress / Completed selectable; persists on both platforms (AC-TASK-08)
US-TASK-007	As a user, I want to change a task's priority, so that I focus on what matters.	Low / Medium / High selectable; persists on both platforms (AC-TASK-08)
US-TASK-008	As a user, I want to search tasks by name, so that I can locate a task.	Partial, case-insensitive; server-side (AC-SRCH-03)
US-TASK-009	As a user, I want to filter tasks by status and priority, so that I can narrow my list.	Individually and combined with search; empty-result message (AC-SRCH-03, 06)

ID	Story	Acceptance criteria
US-DASH-001	As a user, I want a dashboard with Total Projects, Total Tasks, Completed Tasks, Pending Tasks and Projects In Progress, so that I see my overall status.	Five values for my data only; updates after changes (AC-DASH-01..04)
US-MOB-001	As a mobile user, I want to register, log in and log out in the Android app, so that I can use my account on the go.	Same endpoints as web; secure token storage (AC-MOB-02, 08)
US-MOB-002	As a mobile user, I want to view the dashboard, my projects and each project's tasks, so that I can check status anywhere.	AC-MOB-03, 04
US-MOB-003	As a mobile user, I want to create, edit, delete, complete tasks and change status and priority, so that I can manage tasks from my phone.	AC-MOB-05
US-MOB-004	As a mobile user, I want to search and filter tasks, so that I can find tasks on a small screen.	AC-MOB-06
US-MOB-005	As a mobile user, I want pull-to-refresh, so that I can see changes made elsewhere.	AC-MOB-07
US-MOB-006	As a mobile user, I want a clear message when I'm offline instead of a crash, so that I know to retry.	AC-MOB-10
US-SYNC-001	As a user, I want a task I create on web to appear on mobile after refresh, so that both apps agree.	AC-SYNC-01
US-SYNC-002	As a user, I want status changes on mobile to appear on web after refresh, and deletions to propagate, so that data stays consistent.	AC-SYNC-02, 03
US-SEC-001	As a user, I want my data isolated from other users, so that nobody can read or alter it by guessing IDs.	Cross-user matrix returns 404 for every foreign id (AC-SEC-03)

ID	Story	Acceptance criteria
US-SEC-002	As the system owner, I want login brute force throttled, so that accounts are harder to attack.	AC-AUTH-05, AC-SEC-07
US-OPS-001	As an evaluator, I want to run backend, web and mobile from the README, so that I can verify the work.	AC-DOC-01, AC-DEP-01..04
US-OPS-002	As an evaluator, I want API docs and an ER diagram, so that I can review the design.	AC-DOC-02, 03

10. End-to-End User Journeys

#	Journey	Trigger / Preconditions	Steps	Expected result	Error cases
J1	New user registration	Visitor opens Register; no session	1) Enter Full Name, Email Address, Password 2) Client validation 3) Submit 4) Token stored 5) Redirect	Account created, Dashboard (zeros) shown, empty-state hint	Duplicate email → inline An account with this email already exists. ; invalid fields → inline errors; 429 → Too many attempts. Please try again later. ; offline → connectivity message
J2	Returning user login	Visitor has account	1) Open Login 2) Enter credentials 3) Submit	Dashboard with own data	Wrong credentials → Invalid email or password. ; 429; offline
J3	Create project	Logged in; Projects screen	1) Click Create project 2) Fill name (+ optional fields) 3) Save	Project appears in list; toast Project created.	Empty name; end date before start date; 401 → session flow; 500 → generic retry message

#	Journey	Trigger / Preconditions	Steps	Expected result	Error cases
J4	Create task in project	Project exists	1) Open project 2) Add task 3) Fill name, priority, status, due date 4) Save	Task listed under project; counts and dashboard update	Empty name; invalid date; project deleted meanwhile → 404 message This project no longer exists.
J5	Complete task	Task is Pending	1) Tap/click complete control	Status Completed, styled as done, dashboard Completed Tasks +1	Network failure → control reverts, error toast; task deleted elsewhere → 404 message and list refresh
J6	Search / filter tasks	Tasks exist	1) Type in search 2) Choose status 3) Choose priority 4) Clear filters	List narrows by AND of criteria; clear restores full list	No match → No tasks match your search or filters. ; API error → inline retry
J7	Web → Mobile sync	Same account logged in on both	1) Create task on web 2) Mobile pull-to-refresh	Task visible on mobile	Mobile offline → message; refresh later works
J8	Mobile → Web sync	As J7	1) Change status/priority on mobile 2) Refresh web	Web shows updated values	Web session expired → Login with message
J9	Expired token	Token past exp	1) User triggers any request	401 TOKEN_EXPIRED → session cleared → Login screen with Your session has expired. Please log in again.	Parallel requests all fail → handled once; user input in open form is lost (accepted)
J10	No network	Device offline	1) Open app/refresh/submit	Unable to connect. Please check your internet connection and try again. + Retry; no crash or blank screen	Offline during submit → form data preserved; timeout (15 s) treated as offline

#	Journey	Trigger / Preconditions	Steps	Expected result	Error cases
J11	Unauthorized resource access attempt	User B holds A's project/task id	1) B calls GET/PUT/DELETE with A's id (URL or body <code>projectId</code>)	404 <code>*_NOT_FOUND</code> ; nothing read or changed; no information about A	Malformed id → 400; missing token → 401

11. Information Architecture

Web (React SPA).

Authentication (public)

- └ /login
- └ /register

Application (protected, shell with navigation + user menu/logout)

- └ / → Dashboard
- └ /projects → Project List (search, status filter, Create project)
 - └ /projects/new → Create Project
 - └ /projects/:id → Project Details (project info + task list + add task)
 - └ /projects/:id/edit → Edit Project
- └ /tasks → Tasks (all tasks; search, status and priority filters) (task create/edit in a modal dialog or side panel, reachable from Project Details and Tasks)
- * → Not Found page

- Protected routes redirect unauthenticated users to `/login` (remembering the intended path); authenticated users visiting `/login` or `/register` are redirected to `/`.
- Navigation: left sidebar ≥ 1024 px; top bar with menu drawer < 1024 px. User menu holds the user's name and Log out.

Mobile (React Native + React Navigation).

Root

- └ Splash/Loading (session check)
- └ Auth stack: Login ↔ Register
- └ App (bottom tabs)
 - └ Dashboard
 - └ Projects stack: Projects → Project Details → Task Form / Task Details
 - └ Tasks stack: Tasks → Task Details → Task Form
 - └ Profile: user info + Log out

12. UI/UX Requirements

Goal: polished and professional without excess. Prioritize clarity, usability, consistency, accessibility and performance. No decorative animation.

ID	Requirement	Tag
UX-001	Responsive web: fluid layout 320 px → 1920 px; breakpoints < 640, 640–1023, ≥ 1024; no horizontal page scroll (wide tables scroll inside their own container or collapse to cards). Mobile-first CSS.	M (responsive) / R
UX-002	Design tokens defined once and reused: spacing scale 4/8/12/16/24/32; type scale (page title 24–28, section title 18–20, body 16, secondary 14, caption 12); one primary color, neutral grays, semantic colors (success, warning, danger, info). Light theme required; dark theme Could.	R
UX-003	Visual hierarchy: one H1 per page, primary action per screen is the single filled button; secondary actions are outline/text; destructive actions use danger styling and always require confirmation.	R
UX-004	Buttons expose states: default, hover, keyboard focus-visible (2 px ring), active, disabled, loading (spinner + disabled, label retained). Submit controls disable while a request is pending to prevent duplicate submissions.	M (loading indicators) / R
UX-005	Forms: visible labels (not placeholder-only), required marking, inline validation on blur and submit, error text linked via <code>aria-describedby</code> , first invalid field focused on submit, server field errors mapped to fields, character counters for long text.	M (validation) / R
UX-006	Status badges: Not Started/Pending = neutral gray; In Progress = blue; Completed = green. Always text + icon + color.	R
UX-007	Priority indicators: Low = muted (down icon), Medium = amber (dash icon), High = red (up icon), each with text label.	R

ID	Requirement	Tag
UX-008	Search box with clear (×) button and accessible label; filters as select/segmented control (web) and chips/bottom sheet (mobile); active filter count visible; Clear filters control.	R
UX-009	Loading: skeletons for lists and stat cards, spinners for buttons; loading must never leave a blank screen. Slow API (> 400 ms) still shows an indicator.	M
UX-010	Empty states: icon/illustration-free text, one-sentence explanation, primary call to action where relevant (copy in §24).	R
UX-011	Error states: human-readable message, no stack traces or raw codes, Retry where retrying helps; field errors inline; page-level errors in an <code>role=alert</code> region.	M (error handling) / R
UX-012	Success feedback: non-blocking toast/snackbar (auto-dismiss ≈ 4 s, dismissible) for create/update/delete.	R
UX-013	Confirmation dialogs: focus trapped, Esc closes (web), explicit buttons <code>Cancel</code> and <code>Delete</code> (danger), names the object and consequences.	R
UX-014	Long content: names truncate with ellipsis in lists (full text in details/tooltip); descriptions wrap and preserve line breaks; layouts must not break with 120/150-char names or 2000-char descriptions.	R
UX-015	Dates: displayed in a localized readable format (e.g. <code>Oct 7, 2026</code>); date inputs use native pickers; <code>Overdue</code> label per TASK-016.	R
UX-016	Accessibility (WCAG 2.1 AA target): text contrast ≥ 4.5:1, full keyboard operability on web, logical focus order, semantic landmarks (<code>header</code> , <code>nav</code> , <code>main</code>), <code>aria-live</code> for toasts, touch targets ≥ 44 px web / 48 dp mobile, respect <code>prefers-reduced-motion</code> .	R
UX-017	Motion: transitions ≤ 200 ms, only for feedback (focus, dialog open); no parallax or large animations.	R

ID	Requirement	Tag
UX-018	Consistent copy: sentence case, plain language, same terms as the PDF (Project, Task, Status, Priority, Due Date).	R

12.1 Core-First Quality Principle

The core functionality (authentication, projects, tasks, dashboard, search/filtering — on web and Android, synchronized through one backend) is built end-to-end **first** (§40.1), and the best achievable UI/UX is part of that core: it is **P0, not deferred polish** (§44). The fastest route to a professional result is to build on proven open-source design systems and shared tokens instead of hand-styling controls. §12.1–§12.6 are **RECOMMENDED ENGINEERING DECISIONS**, except where they restate the PDF's mandatory UX items (responsive design, component structure, form validation, loading indicators, error handling, clean UX — SRC-081..086, SRC-096). Using libraries and open-source resources is explicitly allowed by the PDF (SRC-200).

12.2 Open-Source UI Foundation

Layer	Web (React + Vite)	Android (React Native + Expo)	License	Why
Styling and tokens	Tailwind CSS with CSS variables from §12.3	React Native Paper theme from the same tokens	MIT	One palette, spacing and type scale on both platforms
Components	shadcn/ui components (source copied into <code>web/src/components/ui</code>) on Radix UI primitives: Button, Input, Textarea, Select, Dialog, AlertDialog, DropdownMenu, Sheet, Tabs, Badge, Card, Progress, Skeleton, Tooltip, Table, Form, Calendar/Popover	React Native Paper (Material Design 3): TextInput, Button, FAB, Chip, SegmentedButtons, Searchbar, Dialog, Snackbar, Card, ProgressBar, Menu, AppBar, List	MIT	Accessible, keyboard-operable, widely used, professional by default; no hand-built dialogs or selects
Icons	lucide-react	<code>@expo/vector-icons</code> (MaterialCommunityIcons)	ISC / MIT	Consistent, open icon sets; no emoji as icons
Typography	Inter, self-hosted via <code>@fontsource</code> (CSP-friendly, <code>font-display: swap</code>)	Inter via <code>expo-font</code> , or the system font (Roboto)	SIL OFL	Clean, legible UI face
Forms and validation	React Hook Form + Zod (shadcn Form)	React Hook Form + Zod	MIT	Same rules as backend; accessible error wiring
Feedback	sonner toasts, AlertDialog confirmations	Paper Snackbar and Dialog	MIT	Consistent success/error/confirm patterns
Dates	date-fns + shadcn Calendar (react-day-picker)	<code>react-native-paper-dates</code> or <code>@react-native-community/datetimestpicker</code>	MIT	Native-feeling date entry

Layer	Web (React + Vite)	Android (React Native + Expo)	License	Why
Navigation	React Router	React Navigation (themed with Paper)	MIT	Standard, well documented
Optional (Should/Could)	dnd-kit for the board view; Recharts only if a chart beyond the CSS status bar is wanted	—	MIT	Only after the core slices pass
Reference designs (study, do not clone)	shadcn/ui example pages (Dashboard, Tasks, Authentication) for layout patterns; Material Design 3 guidance for Android patterns	Material Design 3 guidance	MIT / open guidelines	Proven layouts without copying a product

Governance rules (R):

- Allowed licenses: MIT, ISC, Apache-2.0, BSD, SIL OFL (fonts), CC0 / CC-BY with attribution. No paid, proprietary, scraped or copied commercial designs; no logos, screenshots or branding of commercial products.
- Record every third-party UI source (name, license, link) in `THIRD_PARTY_NOTICES.md` and a short "Design credits" list in the README; keep license headers in copied files.
- Open-source admin/dashboard templates may be used as **reference**; do not import a whole template (dead pages, unneeded dependencies, license traps). Each imported component must serve a requirement in this PRD.
- Style through tokens, not by forking Radix or Paper internals; do not strip ARIA attributes from primitives.
- A library choice must never change the API contract or any **M** requirement; run `npm audit` and commit lockfiles (SEC-017).

12.3 Design Tokens

Defined once (web CSS variables / Tailwind theme; mobile Paper theme). No hard-coded hex values outside the token files.

Token	Light value	Use
<code>background</code>	<code>#FAFAFA</code>	App background
<code>surface</code>	<code>#FFFFFF</code>	Cards, dialogs, inputs
<code>border</code>	<code>#E4E4E7</code>	Borders, dividers
<code>text</code>	<code>#18181B</code>	Primary text
<code>text-muted</code>	<code>#52525B</code>	Secondary text ($\geq 7:1$ on surface)

Token	Light value	Use
primary	#4F46E5 (hover #4338CA , on-primary #FFFFFF)	Primary buttons, links, focus ring ($\approx 6.3:1$ with white)
danger	#DC2626 (on-danger #FFFFFF)	Destructive actions, errors ($\approx 4.8:1$ with white)
success	#15803D	Success text/buttons
warning	#B45309	Warning text
Status: Not Started / Pending	bg #F4F4F5 , text #3F3F46	Neutral badge
Status: In Progress	bg #DBEAFE , text #1E40AF	Blue badge
Status: Completed	bg #DCFCE7 , text #166534	Green badge
Priority: Low	bg #F4F4F5 , text #3F3F46 , down-arrow icon	Muted badge
Priority: Medium	bg #FEF3C7 , text #92400E , minus icon	Amber badge
Priority: High	bg #FEE2E2 , text #991B1B , up-arrow icon	Red badge
Type	Inter 400 / 500 / 600; scale from UX-002 (24–28 / 18–20 / 16 / 14 / 12)	All text
Shape and depth	Radius 8 px (controls), 12 px (cards, dialogs); one subtle shadow 0 1px 2px rgba(0,0,0,0.06) ; 2 px primary focus ring with 2 px offset	All components
Layout	4-px spacing grid; web content max-width 1200 px; mobile horizontal padding 16 dp	All screens
Dark theme (Could)	zinc-950 background, zinc-900 surface, indigo-400 primary; all badge pairs re-derived to $\geq 4.5:1$	Optional

Contrast of every text/background token pair is asserted $\geq 4.5:1$ in a unit test.

12.4 Web Screen Layouts (reference wireframes)

Login / Register (centered card; Register adds Full Name and a link back to Login):

```

+-----+
|                                     |
|           [logo] Project Management System           |
|           +-----+                               |
|           | Welcome back                           |
|           | Log in to your workspace                 |
|           | Email Address                           |
|           | [ name@example.com                      ] |
|           | Password                               |
|           | [ ***** (show) ]                     |
|           | [           Log in           ]          |
|           | No account yet? Create one              |
|           +-----+                               |
|                                     |
+-----+

```

App shell + Dashboard (sidebar ≥ 1024 px; below that a top bar with a drawer):

```

+-----+-----+-----+-----+-----+
| [ ] PMS | Dashboard | Asha Rao [menu v] | | | | | | | | |
|         | Overview of your projects and tasks |
| Dashboard | +-----+ +-----+ +-----+ +-----+ +-----+ |
| Projects  | | Total  | | Total  | | Complet. | | Pending | | Projects |
| Tasks     | | Projects | | Tasks   | | Tasks   | | Tasks   | | In Prog. |
|           | | 3      | | 12     | | 5      | | 4      | | 2      |
|           | +-----+ +-----+ +-----+ +-----+ +-----+ |
|           | Task status |
|           | [====Done 5====][==In progress 3==][===Pending 4===] |
|           | Recent projects | [View all] |
|           | Website Redesign (In Progress) #####----- 1 of 4 done |
|           | Mobile App (Not Started) ----- 0 of 3 done |
| Log out  |
+-----+-----+-----+-----+-----+

```

Project List (cards: 1 / 2 / 3 columns by width):

```

| Projects | [ + New project ] |
| [ Search projects by name... ] [ Status: All v ] Clear filters |
| +-----+ +-----+ +-----+ |
| | Website Redesign [:]| | Mobile App [:]| | Marketing [:]| |
| | (In Progress) | | (Not Started) | | (Completed) | |
| | Oct 1 - Dec 15 | | Nov 1 - Jan 31 | | Sep 1 - Sep 30 | |
| | #####----- 1 of 4 | | ----- 0 of 3 | | ##### 5/5 | |
| +-----+ +-----+ +-----+ |

```

Project Details with tasks (List | Board toggle: Board is UI-016, Should/P1):

```

| Projects / Website Redesign          [ Edit ] [ Delete ] |
| Website Redesign (In Progress)      Oct 1 - Dec 15, 2026 |
| Q4 refresh of the marketing site    |
| Progress #####----- 1 of 4 tasks completed |
| Tasks                               [ List | Board ]      [ + Add task ] |
| [ Search tasks... ] [ Status: All v ] [ Priority: All v ] Clear |
| [ ] Design REST API (High) [ In Progress v ] Due Oct 20 [:]|
| [x] Set up database (Medium) [ Completed v ] Due Oct 10 [:]|
| [ ] Write README (Low) [ Pending v ] Due Oct 30 [:]|

```

Task dialog (create/edit):

```

+-----+
| Add task                                [x] |
| Task Name *      [                      ] |
| Description      [                      ] |
| Priority          (Low)(Medium)(High)|
| Status           [ Pending v ] |
| Due Date         [ Pick a date v ] |
|                  [Cancel] [Create] |
+-----+

```

The global **Tasks** page reuses the task toolbar and rows and adds a Project column; below 640 px task rows become stacked cards (name, badges, due date).

12.5 Android Screen Layouts (reference wireframes)

Dashboard	Tasks	Task details
Dashboard Total Progress Projects 3 Tasks 12 Redesign Completed 5 Pending 4 Priority [v] Projects In Progress 2 Dash Proj Tasks Profile	Tasks [Search tasks...] Status: (All)(Pending) (In progress)(Done) Priority: (All)(Low) (Medium)(High) [] Design REST API High - In Progress [x] Set up database Medium - Completed (+) Dash Proj Tasks Profile	< Task Design REST API [High] [In Progress] Due Oct 20, 2026 Website Description [] Mark Status [v] [Edit] [Delete]

Projects: Searchbar + status chips, project cards with progress bar; Project Details: header card + the same task list and filters as Tasks, with a floating Add task button; Task form: full-screen with sticky Save button above the keyboard.

12.6 UI Quality Requirements

ID	Requirement	Priority / Tag
UI-001	Build with the §12.2 stack; do not re-implement dialogs, selects, date pickers, toasts or menus by hand.	Must / R

ID	Requirement	Priority / Tag
UI-002	All colors, radii, spacing and type come from the §12.3 tokens, shared by web and mobile; no hard-coded hex outside token files.	Must / R
UI-003	Web app shell: sidebar $\geq$ 1024 px, top bar + drawer below; active nav state; user menu with name and Log out; page header pattern (title, one-line description, primary action on the right).	Must / R
UI-004	Auth screens: centered card, labeled fields, show/hide password, inline errors, loading submit button, alternate-screen link, autofocus on the first field, Enter submits.	Must / R
UI-005	Dashboard: five stat cards (icon, number, PDF label) in the PDF order; task-status breakdown bar built from <code>pendingTasks</code> , <code>inProgressTasks</code> , <code>completedTasks</code> (CSS/SVG, with text legend); recent projects (first five of <code>GET /api/projects</code> , client-side). The breakdown and recent list are Should; the five cards are Must.	Must / Should
UI-006	Project list: responsive card grid with status badge, date range, progress bar (<code>completedTaskCount</code> / <code>taskCount</code>), kebab menu (Edit, Delete); toolbar with search, status filter and Clear filters.	Must / R
UI-007	Project details: header, description, progress, task toolbar, task rows with complete checkbox, inline status and priority selects, due date, kebab (Edit, Delete).	Must / R
UI-008	Task form: Task Name, Description, Priority (segmented Low/Medium/High), Status, Due Date; dialog on web, full screen on mobile; project selector only when creating from the global Tasks page.	Must / R
UI-009	Quick actions (complete, status, priority) update the UI immediately (optimistic) and revert with an error toast if the request fails; dashboard and lists are invalidated afterwards.	Must / R
UI-010	Skeletons mirror the final layout (no layout shift) and appear only after a 150 ms delay to avoid flicker.	Must / R

ID	Requirement	Priority / Tag
UI-011	Empty and filtered-empty states per §24, each with one clear action.	Must / R
UI-012	Feedback: sonner/Snackbar for success and errors (error toasts stay longer and offer Retry where useful); AlertDialog/Dialog for every destructive action.	Must / R
UI-013	Keyboard and focus (web): Esc closes dialogs, focus returns to the trigger, visible focus ring, logical tab order; / focuses search (Could).	Must / R
UI-014	Task rows collapse into stacked cards below 640 px; no horizontal page scroll at any width $\geq$ 320 px.	Must / R
UI-015	Dark theme following prefers-color-scheme with a toggle in the user menu.	Could / R
UI-016	Board view toggle in Project Details: columns Pending / In Progress / Completed; drag a card (dnd-kit) or use its menu to change status through PUT /api/tasks/{id} ; a keyboard-accessible alternative is required.	Should (P1) / R
UI-017	document.title per route, favicon and app icon, consistent wordmark.	Should / R
UI-018	Performance: lazy-loaded route chunks, self-hosted fonts; Lighthouse Performance and Accessibility $\geq$ 90 on Login and Dashboard in a production build.	Should / R
UI-019	Accessibility: Radix/Paper ARIA retained; axe-core reports zero serious or critical violations on Login, Dashboard, Project List, Project Details and Tasks.	Must / R
UI-020	Motion: only feedback transitions $\leq$ 200 ms; honor prefers-reduced-motion .	Must / R
UI-M01	Android uses a Paper (Material Design 3) theme built from the §12.3 tokens; Paper components for inputs, buttons, chips, dialogs, snackbars, cards, menus.	Must / R
UI-M02	Bottom tab bar with icons and labels: Dashboard, Projects, Tasks, Profile.	Must / R

ID	Requirement	Priority / Tag
UI-M03	Dashboard: two-column stat grid (fifth card full width), pull-to-refresh, optional status bar.	Must / R
UI-M04	Lists: <code>FlatList</code> with rows ≥ 56 dp, complete via checkbox tap, kebab menu for edit/delete.	Must / R
UI-M05	Filters: Searchbar pinned on top plus horizontally scrollable chips for Status (All, Pending, In Progress, Completed) and Priority (All, Low, Medium, High); active state visible; Clear.	Must / R
UI-M06	Task form: full screen, sticky Save above the keyboard, native date picker, segmented Priority.	Must / R
UI-M07	Floating Add task button on Project Details and Tasks; Snackbar feedback; Dialog for delete confirmation.	Must / R
UI-M08	Branded splash with progress indicator during the session check; adaptive icon.	Should / R
UI-M09	Layouts remain intact at 130% system font scale; touch targets ≥ 48 dp.	Must / R
UI-M10	Performance: memoized rows and stable keys; first content visible within 2 s after login on a mid-range device with a warm API.	Should / R

AC ID	Acceptance criterion (measurable)	Test method	Pri
AC-UI-01	Code review: web UI uses the §12.2 stack, a token file exists, and no hex colors appear outside token files and SVG assets; mobile uses a Paper theme from the same tokens	Code review + grep	Must
AC-UI-02	Login, Dashboard, Project List, Project Details, Tasks and the task dialog match the §12.4 layouts at 360, 768 and 1280 px with no horizontal page scroll; Android screens match §12.5	Manual checklist + screenshots	Must

AC ID	Acceptance criterion (measurable)	Test method	Pri
AC-UI-03	axe-core: zero serious/critical violations on the five key web routes; all status/priority/text token pairs $\geq 4.5:1$; Lighthouse Accessibility ≥ 90 on Login and Dashboard	Automated	Must
AC-UI-04	Quick actions update immediately and revert with a toast on failure (simulated 500); skeletons cause no visible layout shift on Dashboard and lists	Component test + manual	Must
AC-UI-05	On a physical Android device: touch targets ≥ 48 dp, layouts intact at 130% font scale, bottom tabs and FAB work one-handed	Manual device check	Must
AC-UI-06	First-time-user walkthrough on web and on Android without help: register, create a project, add a task, complete it, find it with a filter — each in under 3 minutes (time recorded)	Manual script	Should

AC-UI-01..06 extend SRC-086 and SRC-096 (clean UX) in §38.2; TC-UI-nn mirrors AC-UI-nn.

13. Web Application Requirements

The PDF requires: React or Next.js (React chosen), responsive design, proper component structure, form validation, loading indicators, error handling, clean user experience (all **M**). Everything below that names a library or structure is a **RECOMMENDED ENGINEERING DECISION** unless it restates those items.

ID	Requirement	Tag
WEB-001	React + Vite + TypeScript (<code>strict: true</code>). ESLint + Prettier. Output is a static SPA.	M (React) / R
WEB-002	Structure under <code>/web/src</code> : <code>app/</code> (providers, router), <code>pages/</code> (route components), <code>features/{auth,projects,tasks,dashboard}/</code> (components, hooks, api, schemas), <code>components/ui/</code> (design-system primitives), <code>lib/</code> (api client, format, storage), <code>types/</code> .	M (component structure) / R

ID	Requirement	Tag
WEB-003	Reusable components: Button, Input, Textarea, Select, DateInput, Badge (status/priority), Modal, ConfirmDialog, Toast, Spinner, Skeleton, EmptyState, ErrorState, StatCard, SearchInput, FilterBar, ProjectForm, TaskForm, ProjectCard/Row, TaskRow, AppShell. Guideline: components over ~200 lines are split; no giant page components; no business rules in JSX.	M / R
WEB-004	Routing with React Router per §11; route-level code splitting; custom Not Found page.	R
WEB-005	Auth state in an <code>AuthProvider</code> (status: <code>loading</code> <code>authenticated</code> <code>unauthenticated</code> ; <code>user</code> ; <code>login</code> , <code>register</code> , <code>logout</code>). <code>ProtectedRoute</code> blocks rendering of protected pages until status is known (no flash of protected content) and redirects to <code>/login</code> .	M (protected routes) / R
WEB-006	One Axios instance in <code>lib/api</code> : <code>baseUrl = import.meta.env.VITE_API_URL</code> , 15 s timeout, request interceptor adds <code>Authorization: Bearer <token></code> , response interceptor normalizes every failure into <code>AppError { status, code, message, fieldErrors?, isNetworkError }</code> and on 401 <code>TOKEN_EXPIRED</code> / <code>TOKEN_INVALID</code> clears the session and navigates to <code>/login</code> with the expiry message. Components never call Axios directly.	R
WEB-007	Server state with TanStack Query (query keys per resource; mutations invalidate affected lists, the dashboard, and details; <code>retry: 1</code> for network errors, none for 4xx). Auth/UI state in React Context. No Redux.	R
WEB-008	Forms use React Hook Form + Zod resolvers. Zod schemas mirror backend rules (same limits/enums). Backend validation errors (<code>details[]</code>) are mapped onto fields.	M (form validation) / R
WEB-009	A top-level and a route-level Error Boundary render a friendly fallback with a reload action; a global <code>unhandledrejection</code> handler logs and shows a toast. No blank screens, no unhandled promise rejections in app code.	M (error handling) / R

ID	Requirement	Tag
WEB-010	Token persistence: <code>localStorage</code> key <code>pms.token</code> (OD-06). Mitigations: React escaping only (no <code>dangerouslySetInnerHTML</code>), strict CSP on the host, no third-party scripts, dependency audit.	R
WEB-011	Dashboard, Project, Task and Search/Filter screens implement the requirements in §8B-8E and the states in §24. Search/filter state lives in the URL query.	M
WEB-012	Supported browsers: latest two versions of Chrome, Edge, Firefox and Safari.	R
WEB-013	Build/deploy: <code>VITE_API_URL</code> provided at build time; host configured with SPA fallback (<code>/*</code> → <code>/index.html</code>) and security headers (CSP restricting <code>connect-src</code> to the API origin, <code>X-Content-Type-Options</code> , <code>Referrer-Policy</code>).	R

14. Mobile Application Requirements

The PDF requires: React Native (Expo or bare) or Flutter (React Native + Expo chosen), proper navigation and screen structure, form validation, loading indicators and pull-to-refresh, error handling (including no network and expired login), secure token storage, clean UX on a phone screen (all **M**).

ID	Requirement	Tag
MAPP-001	Expo (managed workflow, current stable SDK) + React Native + TypeScript strict. Android required; iOS optional.	M / R
MAPP-002	Structure under <code>/mobile/src</code> : <code>navigation/</code> , <code>screens/</code> , <code>components/</code> , <code>features/{auth,projects,tasks,dashboard}/</code> , <code>api/</code> , <code>auth/</code> , <code>hooks/</code> , <code>lib/</code> , <code>theme/</code> .	M (screen structure) / R
MAPP-003	Navigation: React Navigation (native stack + bottom tabs) per §11; auth gating decided at the root from the auth status.	M / R

ID	Requirement	Tag
MAPP-004	API service mirrors web: one Axios instance, <code>baseUrl</code> from <code>EXPO_PUBLIC_API_BASE_URL</code> , 15 s timeout, Bearer interceptor, normalized <code>AppError</code> , 401 handler. The same endpoint paths, payloads and enums as web.	M / R
MAPP-005	Auth state: <code>AuthProvider</code> loads the token from <code>expo-secure-store</code> at start, keeps it in memory for requests, validates via <code>/api/auth/me</code> , deletes it on logout/expiry. SecureStore key <code>pms_token</code> .	M / R
MAPP-006	Server state with TanStack Query; list screens use <code>FlatList</code> with <code>RefreshControl</code> calling <code>refetch()</code> (pull-to-refresh) and show <code>isRefetching</code> state.	M (pull-to-refresh) / R
MAPP-007	Network handling: Axios network errors/timeouts map to <code>isNetworkError</code> ; screens render the offline message with Retry; optional <code>@react-native-community/netinfo</code> banner.	M / R
MAPP-008	Forms: React Hook Form + Zod; keyboard-aware scrolling; correct <code>keyboardType</code> , <code>autoComplete</code> , <code>secureTextEntry</code> ; native date picker.	M / R
MAPP-009	Loading: skeleton/spinner on first load, button spinners, refresh control; no blank screens.	M
MAPP-010	Android: stable <code>android.package</code> id; adaptive icon and splash; handle hardware Back via navigation; respect safe areas/edge-to-edge; cleartext HTTP disabled in release (dev-only exception, e.g. <code>10.0.2.2</code> emulator host); only network permissions.	R
MAPP-011	Build: EAS Build <code>preview</code> profile producing an installable <code>.apk</code> (or Firebase App Distribution link) pointing at the deployed backend; build-time env supplies the API URL.	M (distribution) / R
MAPP-012	Reusable components: Button, TextField, StatCard, StatusBadge, PriorityBadge, ProjectListItem, TaskListItem, EmptyState, ErrorState, LoadingView, FilterChips, ConfirmAlert, ScreenContainer.	R

15. Backend Architecture

PDF requirements (**M**): Node.js with Express or NestJS (Express chosen); one backend for web and mobile; REST architecture; proper route organization; middleware usage; error handling; logging; clean code structure; CORS configured for the web app's domain. Modular monolith: one deployable, one database.

```

flowchart LR
    subgraph Clients
        WEB[React web SPA - Vite]
        AND[Android app - React Native Expo]
    end
    WEB -->|HTTPS REST JSON + Bearer JWT| API
    AND -->|HTTPS REST JSON + Bearer JWT| API
    subgraph Backend
        API[Express API - modular monolith]
    end
    API -->|Prisma ORM - parameterized SQL| DB[(PostgreSQL)]

```

```

flowchart TD
    R[Incoming request] --> ID[Request ID + structured logger]
    ID --> SEC[helmet + CORS + body size limit]
    SEC --> T{Route type}
    T -->|register / login| LIM[Rate limiter]
    T -->|protected| AUTHN[authenticate: verify JWT, load user id]
    LIM --> VAL[Zod validation: params, query, body]
    AUTHN -->|invalid or expired| E401[401 via error middleware]
    AUTHN --> VAL
    VAL -->|invalid| E400[400 via error middleware]
    VAL --> CTRL[Controller: HTTP in and out only]
    CTRL --> SVC[Service: business rules + ownership]
    SVC --> REPO[Data access: Prisma]
    REPO --> DB[(PostgreSQL)]
    SVC -->|missing or not owned| E404[404 via error middleware]
    CTRL --> RES[JSON response]

```

Layers (RECOMMENDED): Routes → Middleware → Controllers → Services → Data access (Prisma) → PostgreSQL.

Layer	Responsibility	Must not
Routes (<code>src/routes/*.routes.ts</code>)	Map method+path to middleware chain and controller; one file per resource; mounted under <code>/api</code>	Contain business logic or queries

Layer	Responsibility	Must not
Middleware	<code>requestId</code> , <code>httpLogger</code> (<code>pino-http</code>), <code>helmet</code> , <code>cors</code> , <code>express.json</code> (<code>{ limit: '100kb' }</code>), <code>authLimiter</code> , <code>authenticate</code> , <code>validate(schema)</code> , <code>notFound</code> , <code>errorHandler</code>	Swallow errors
Validation (<code>src/schemas/*.ts</code>)	Zod schemas for params/query/body (strict objects, trimmed strings, enums, dates)	Trust client types
Authentication middleware	Parse <code>Authorization: Bearer</code> , verify signature/expiry, confirm user exists, set <code>req.userId</code> ; map failures to <code>UNAUTHENTICATED</code> , <code>TOKEN_INVALID</code> , <code>TOKEN_EXPIRED</code>	Read user id from body/query
Controllers	Translate validated request → service call → HTTP status/JSON	Access Prisma directly
Services (<code>src/services/*.service.ts</code>)	Business rules, ownership enforcement (<code>ownerId</code> passed into every call), transactions, domain errors (<code>NotFoundError</code> , <code>ConflictError</code>)	Know about Express <code>req/res</code>
Data access (<code>src/repositories/*</code> or service-level Prisma calls)	Prisma queries always including the owner predicate; no raw string SQL	Expose <code>passwordHash</code> outside the auth service
Error middleware	Map domain/validation/JWT/Prisma errors to the standard envelope (§18.1); hide internals in production; log with request id	Leak stack traces in production
Logging	pino JSON logs; levels by env; log method, path, status, duration, request id, user id; redact <code>authorization</code> , <code>password</code> , <code>passwordHash</code> , <code>token</code>	Log request bodies of auth routes

Layer	Responsibility	Must not
Rate limiting	<code>express-rate-limit</code> on register and login (AUTH-013); in-memory store acceptable for single instance (note: use a shared store if scaled out)	Be applied only on the client
Config (<code>src/config.ts</code>)	Parse and validate env with Zod at boot; fail fast if <code>JWT_SECRET</code> / <code>DATABASE_URL</code> missing or weak	Read <code>process.env</code> elsewhere

Recommended layout:

`/backend/src/{app.ts,server.ts,config.ts,routes,controllers,services,repositories,schemas,middleware,errors,utils}` ,

`/backend/prisma/{schema.prisma,migrations,seed.ts}` , `/backend/tests . app.ts` exports the Express app (no `listen`) so Supertest can import it. Additional recommended route: `GET /api/health` (public; returns `{ data: { status: 'ok' } }`) and checks DB connectivity) for deployment health checks; API docs at `GET /api/docs` .

16. Database Design

PostgreSQL (PDF: PostgreSQL or MySQL — PostgreSQL chosen). Required (**M**): proper relational design, foreign key relationships, normalized structure.

Relationships. `USER 1 — N PROJECT` and `PROJECT 1 — N TASK` . A task has no direct `user_id` ; its owner is derived through its project (single source of ownership, no redundancy).

```

erDiagram
    USER ||--o{ PROJECT : owns
    PROJECT ||--o{ TASK : contains
    USER {
        uuid id PK
        varchar full_name
        varchar email UK
        varchar password_hash
        timestampz created_at
        timestampz updated_at
    }
    PROJECT {
        uuid id PK
        uuid owner_id FK

```

```

    varchar name
    text description
    enum status
    date start_date
    date end_date
    timestamptz created_at
    timestamptz updated_at
}
TASK {
    uuid id PK
    uuid project_id FK
    varchar name
    text description
    enum priority
    enum status
    date due_date
    timestamptz created_at
    timestamptz updated_at
}

```

Entity	Field	Type	Required	Key	Constraints	Description
users	id	uuid	Yes	PK	default gen_random_uuid()	User id
users	full_name	varchar(100)	Yes		CHECK length(trim) ≥ 1	Full Name
users	email	varchar(254)	Yes	UNIQUE	stored lowercase	Email Address
users	password_hash	varchar(100)	Yes		bcrypt hash only	Never exposed
users	created_at / updated_at	timestamptz	Yes		default now()	Audit timestamps
projects	id	uuid	Yes	PK		Project id
projects	owner_id	uuid	Yes	FK → users.id	ON DELETE RESTRICT (no user deletion in scope)	Owner
projects	name	varchar(120)	Yes		CHECK length(trim) ≥ 1	Project Name
projects	description	text	Yes		default ""	Description
projects	status	enum project_status	Yes		default NOT_STARTED	Status
projects	start_date / end_date	date	No		CHECK end_date ≥ start_date when both set	Start Date / End Date

Entity	Field	Type	Required	Key	Constraints	Description
projects	created_at / updated_at	timestampz	Yes		default now()	Created Date
tasks	id	uuid	Yes	PK		Task id
tasks	project_id	uuid	Yes	FK → projects.id	ON DELETE CASCADE (OD-02)	Parent project
tasks	name	varchar(150)	Yes		CHECK length(trim) ≥ 1	Task Name
tasks	description	text	Yes		default ""	Description
tasks	priority	enum task_priority	Yes		default MEDIUM	Priority
tasks	status	enum task_status	Yes		default PENDING	Status
tasks	due_date	date	No			Due Date
tasks	created_at / updated_at	timestampz	Yes		default now()	Created Date

Enums: `project_status` = NOT_STARTED, IN_PROGRESS, COMPLETED; `task_status` = PENDING, IN_PROGRESS, COMPLETED; `task_priority` = LOW, MEDIUM, HIGH. **Indexes (RECOMMENDED):** `users(email) unique`; `projects(owner_id, created_at DESC)`; `projects(owner_id, status)`; `tasks(project_id, created_at DESC)`; `tasks(project_id, status)`; `tasks(project_id, priority)`. Optional (Could): `pg_trgm` GIN index on names if search latency grows. **Normalization:** 3NF — no stored counts or derived statuses; counts computed by aggregate queries; enums are database types. **CHECK constraints** (not expressible in Prisma schema) are added in a hand-written SQL migration (R).

Prisma schema excerpt (RECOMMENDED):

```
enum ProjectStatus { NOT_STARTED IN_PROGRESS COMPLETED }
enum TaskStatus    { PENDING IN_PROGRESS COMPLETED }
enum TaskPriority   { LOW MEDIUM HIGH }

model User {
  id          String    @id @default(uuid()) @db.Uuid
  fullName    String    @map("full_name") @db.VarChar(100)
  email       String    @unique @db.VarChar(254)
  passwordHash String    @map("password_hash") @db.VarChar(100)
  createdAt   DateTime   @default(now()) @map("created_at")
  @db.Timestampz(6)
  updatedAt   DateTime   @updatedAt @map("updated_at") @db.Timestampz(6)
  projects    Project[]
  @@map("users")
}
```

```

}
model Project {
  id          String          @id @default(uuid()) @db.Uuid
  ownerId     String          @map("owner_id") @db.Uuid
  name        String          @db.VarChar(120)
  description  String          @default("")
  status       ProjectStatus  @default(NOT_STARTED)
  startDate    DateTime?      @map("start_date") @db.Date
  endDate      DateTime?      @map("end_date") @db.Date
  createdAt    DateTime        @default(now()) @map("created_at")
                                @db.Timestamptz(6)
  updatedAt    DateTime        @updatedAt @map("updated_at") @db.Timestamptz(6)
  owner        User            @relation(fields: [ownerId], references: [id],
onDelete: Restrict)
  tasks        Task[]
  @@index([ownerId, createdAt(sort: Desc)])
  @@index([ownerId, status])
  @@map("projects")
}
model Task {
  id          String          @id @default(uuid()) @db.Uuid
  projectId    String          @map("project_id") @db.Uuid
  name         String          @db.VarChar(150)
  description  String          @default("")
  priority     TaskPriority    @default(MEDIUM)
  status       TaskStatus     @default(PENDING)
  dueDate      DateTime?      @map("due_date") @db.Date
  createdAt    DateTime        @default(now()) @map("created_at")
                                @db.Timestamptz(6)
  updatedAt    DateTime        @updatedAt @map("updated_at") @db.Timestamptz(6)
  project      Project         @relation(fields: [projectId], references: [id],
onDelete: Cascade)
  @@index([projectId, createdAt(sort: Desc)])
  @@index([projectId, status])
  @@index([projectId, priority])
  @@map("tasks")
}

```

17. Database Security

ID	Requirement	Tag
DBS-001	No plaintext passwords: only bcrypt hashes in <code>users.password_hash</code> .	M

ID	Requirement	Tag
DBS-002	All access through Prisma Client methods or parameterized tagged-template <code>\$queryRaw</code> , <code>\$queryRawUnsafe</code> , <code>\$executeRawUnsafe</code> and string-built SQL are forbidden; CI/lint rule or review check enforces it.	M (SQL injection protection) / R (enforcement)
DBS-003	Foreign keys enforce referential integrity; orphan tasks are impossible; unique email enforced by the database.	M
DBS-004	Least privilege: the runtime database role has only SELECT/INSERT/UPDATE/DELETE on application tables; migrations run with a separate privileged role/step where the host allows it.	R
DBS-005	<code>DATABASE_URL</code> and <code>JWT_SECRET</code> come from environment variables; <code>.env</code> is git-ignored; only <code>.env.example</code> is committed with placeholders. TLS to the database (<code>sslmode=require</code>) in production.	M (secure env vars) / R
DBS-006	Safe migrations: Prisma Migrate with committed migration files; <code>migrate dev</code> locally, <code>migrate deploy</code> in CI/production; never <code>db push</code> against production; destructive changes reviewed; backups enabled on the managed database.	R
DBS-007	Seed script creates only synthetic demo data (e.g. <code>demo@example.com</code>) and is never run automatically in production.	M (test data only) / R

18. API Specification

The 15 endpoints below are the assignment's **minimum endpoint set (M)**; paths MUST NOT change. Web and mobile MUST both use them (API-001). Query parameters, response shapes and error codes are **RECOMMENDED ENGINEERING DECISIONS** that fill what the PDF leaves open.

18.1 Conventions

- Base path `/api`. JSON in and out (UTF-8). Property names camelCase. IDs are UUID strings. Enums UPPER_SNAKE_CASE. Dates: YYYY-MM-DD (date-only fields) and ISO-8601 UTC (`createdAt`, `updatedAt`, `expiresAt`).
- **Request headers:** `Accept: application/json`; `Content-Type: application/json` when a body is sent; `Authorization: Bearer <JWT>` on protected endpoints.
Response headers: `Content-Type: application/json; charset=utf-8`, `X-Request-Id`, `Cache-Control: no-store`.
- **Success envelope:** `{ "data": <object | array> }`. 204 responses have no body.
- **Error envelope:** `{ "error": { "code": "<CODE>", "message": "<safe human text>", "details": [ { "field": "<name>", "message": "<text>" } ], "requestId": "<id>" } }` (details only for validation errors).
- Request bodies use strict schemas: unknown properties → 400. Path `id` must be a UUID else 400. Empty body where one is required → 400.
- **Shared shapes.**
 - `User = { id, fullName, email, createdAt }`.
 - `Project = { id, name, description, status, startDate, endDate, createdAt, updatedAt, taskCount, completedTaskCount }`.
 - `Task = { id, projectId, name, description, priority, status, dueDate, createdAt, updatedAt, project: { id, name } }`.
- **Error codes:** | Code | HTTP | Meaning | ---|---|---| | VALIDATION_ERROR | 400 | Body, query or path failed validation | | UNAUTHENTICATED | 401 | No/malformed `Authorization` header | | TOKEN_INVALID | 401 | Bad signature, unknown user, malformed JWT | | TOKEN_EXPIRED | 401 | JWT `exp` passed | | INVALID_CREDENTIALS | 401 | Login failed (generic) | | FORBIDDEN | 403 | Reserved (RBAC bonus); unused in v1 | | PROJECT_NOT_FOUND | 404 | Project missing or not owned by caller | | TASK_NOT_FOUND | 404 | Task missing or its project not owned by caller | | NOT_FOUND | 404 | Unknown route | | EMAIL_ALREADY_REGISTERED | 409 | Email uniqueness conflict | | PAYLOAD_TOO_LARGE | 413 | Body over 100 kb | | RATE_LIMITED | 429 | Auth rate limit exceeded (`Retry-After` set) | | INTERNAL_ERROR | 500 | Unexpected failure; generic message | | SERVICE_UNAVAILABLE | 503 | Database unreachable (R) |

18.2 Authentication endpoints

API-01 • **POST `/api/auth/register`** — *Purpose:* create an account. *Auth:* none (public); rate limited. *Authorization:* n/a. *Path/query params:* none. *Headers:* `Content-Type`. *Body:* `fullName`, `email`, `password` (all required). *Validation:* AUTH-001..005. *Success:* 201 {

`data: { user, token, expiresAt } }`. *Errors:* 400 `VALIDATION_ERROR`; 409 `EMAIL_ALREADY_REGISTERED`; 429; 500. *Security:* bcrypt hash; password never logged/returned; rate limit; generic 500 text.

```
POST /api/auth/register
```

```
{"fullName":"Asha Rao","email":"asha@example.com","password":"Passw0rd!x"}
```

```
201 Created
```

```
{"data":{"user":{"id":"3b1f6c2e-9a4d-4e57-8f0a-5c2d7e9b1a44","fullName":"Asha Rao","email":"asha@example.com","createdAt":"2026-10-07T09:30:00.000Z"},"token":"<jwt>","expiresAt":"2026-10-08T09:30:00.000Z"}}
```

```
409 Conflict
```

```
{"error":{"code":"EMAIL_ALREADY_REGISTERED","message":"An account with this email already exists.","requestId":"req_8c1f"}}
```

API-02 · POST /api/auth/login — *Purpose:* authenticate and obtain a JWT. *Auth:* none; rate limited (10 failed/15 min/IP). *Body:* email, password (required). *Validation:* both present and non-empty; email format. *Success:* 200 `{ data: { user, token, expiresAt } }`. *Errors:* 400; 401 `INVALID_CREDENTIALS` (identical for unknown email and wrong password); 429 with `Retry-After`; 500. *Security:* constant-ish timing (dummy hash), generic message, no account enumeration.

```
POST /api/auth/login
```

```
{"email":"asha@example.com","password":"Passw0rd!x"}
```

```
200 OK
```

```
{"data":{"user":{"id":"3b1f6c2e-9a4d-4e57-8f0a-5c2d7e9b1a44","fullName":"Asha Rao","email":"asha@example.com","createdAt":"2026-10-07T09:30:00.000Z"},"token":"<jwt>","expiresAt":"2026-10-08T09:30:00.000Z"}}
```

API-03 · POST /api/auth/logout — *Purpose:* end the session (stateless; clients discard token). *Auth:* optional — never fails on a missing/expired token. *Body:* none. *Success:* 204 always. *Errors:* 429/500 only. *Security:* no server-side revocation (OD-03); recommended to log the event with user id when a valid token is present.

```
POST /api/auth/logout
```

```
Authorization: Bearer <jwt>
```

```
204 No Content
```

API-04 · GET /api/auth/me — *Purpose:* return the current user (session validation on app start). *Auth:* required. *Authorization:* returns only the token's own user. *Success:* 200 `{ data: { user } }`. *Errors:* 401 `UNAUTHENTICATED` / `TOKEN_INVALID` / `TOKEN_EXPIRED`.

Security: never returns `passwordHash` .

```
GET /api/auth/me
Authorization: Bearer <jwt>

200 OK
{"data":{"user":{"id":"3b1f6c2e-9a4d-4e57-8f0a-5c2d7e9b1a44","fullName":"Asha Rao","email":"asha@example.com","createdAt":"2026-10-07T09:30:00.000Z"}}}
```

18.3 Project endpoints (all require authentication; ownership enforced; foreign/missing id → 404 `PROJECT_NOT_FOUND`)

API-05 · GET `/api/projects` — *Purpose:* list the caller's projects. *Query:* `search` (string ≤ 100, name substring, case-insensitive), `status` (`NOT_STARTED`|`IN_PROGRESS`|`COMPLETED`). *Validation:* SRCH-004..006. *Success:* 200 { `data: Project[]` } ordered by `createdAt` desc (empty array when none). *Errors:* 400, 401, 500. *Security:* `WHERE ownerId = caller` .

```
GET /api/projects?search=web&status=IN_PROGRESS

200 OK
{"data":[{"id":"8f14e45f-ceea-467a-9575-1c1a5b3e2d10","name":"Website Redesign","description":"Q4 refresh","status":"IN_PROGRESS","startDate":"2026-10-01","endDate":"2026-12-15","createdAt":"2026-10-07T09:40:00.000Z","updatedAt":"2026-10-07T10:05:00.000Z","taskCount":4,"completedTaskCount":1}]}
```

API-06 · GET `/api/projects/{id}` — *Purpose:* project details. *Path:* `id` UUID. *Success:* 200 { `data: Project` } . *Errors:* 400 malformed id, 401, 404. *Security:* owner predicate in the query; same 404 for foreign and missing.

```
GET /api/projects/8f14e45f-ceea-467a-9575-1c1a5b3e2d10

404 Not Found (foreign or missing)
{"error":{"code":"PROJECT_NOT_FOUND","message":"Project not found.","requestId":"req_51aa"}}
```

API-07 · POST `/api/projects` — *Purpose:* create a project owned by the caller. *Body:* `name` (required), `description`, `status`, `startDate`, `endDate` . *Validation:* PROJ-001..005. *Success:* 201 { `data: Project` } . *Errors:* 400, 401, 500. *Security:* `ownerId` from JWT only; strict schema rejects `ownerId` / `id` .

```
POST /api/projects
```

```
{"name":"Website Redesign","description":"Q4 refresh","status":"NOT_STARTED","startDate":"2026-10-01","endDate":"2026-12-15"}
```

```
201 Created
```

```
{"data":{"id":"8f14e45f-ceea-467a-9575-1c1a5b3e2d10","name":"Website Redesign","description":"Q4 refresh","status":"NOT_STARTED","startDate":"2026-10-01","endDate":"2026-12-15","createdAt":"2026-10-07T09:40:00.000Z","updatedAt":"2026-10-07T09:40:00.000Z","taskCount":0,"completedTaskCount":0}}
```

```
400 Bad Request
```

```
{"error":{"code":"VALIDATION_ERROR","message":"Request validation failed.","details":[{"field":"endDate","message":"End date must be on or after start date."}], "requestId":"req_2b7a"}}
```

API-08 · PUT /api/projects/{id} — *Purpose:* update a project (partial semantics, OD-07). *Body:* any subset of `name`, `description`, `status`, `startDate`, `endDate`; ≥ 1 field. *Validation:* as create; date ordering on merged values. *Success:* 200 { `data: Project` }. *Errors:* 400, 401, 404. *Security:* owner predicate on the update itself (`updateMany` where `id+ownerId` or read-then-write in a transaction).

```
PUT /api/projects/8f14e45f-ceea-467a-9575-1c1a5b3e2d10
```

```
{"status":"IN_PROGRESS"}
```

```
200 OK
```

```
{"data":{"id":"8f14e45f-ceea-467a-9575-1c1a5b3e2d10","name":"Website Redesign","description":"Q4 refresh","status":"IN_PROGRESS","startDate":"2026-10-01","endDate":"2026-12-15","createdAt":"2026-10-07T09:40:00.000Z","updatedAt":"2026-10-07T10:05:00.000Z","taskCount":0,"completedTaskCount":0}}
```

API-09 · DELETE /api/projects/{id} — *Purpose:* delete a project and, by cascade, its tasks (OD-02). *Success:* 204. *Errors:* 400, 401, 404. *Security:* owner predicate; single transaction.

```
DELETE /api/projects/8f14e45f-ceea-467a-9575-1c1a5b3e2d10
```

```
204 No Content
```

18.4 Task endpoints (all require authentication; ownership resolved through the task's project)

API-10 · GET /api/tasks — *Purpose:* list the caller's tasks, optionally for one project.

Query: `projectId` (UUID), `search` (name substring, case-insensitive, ≤ 100), `status` (`PENDING|IN_PROGRESS|COMPLETED`), `priority` (`LOW|MEDIUM|HIGH`); combined with `AND`. *Success:* 200 { `data`: Task[] } by `createdAt` desc. *Errors:* 400; 401; 404 `PROJECT_NOT_FOUND` when `projectId` is not owned/missing. *Security:* `WHERE project.ownerId = caller`.

```
GET /api/tasks?projectId=8f14e45f-ceeda-467a-9575-1c1a5b3e2d10&search=api&status=IN_PROGRESS&priority=HIGH
```

```
200 OK
{"data":[{"id":"c4ca4238-a0b9-4f23-8e51-9d1b2f6a7e33","projectId":"8f14e45f-ceeda-467a-9575-1c1a5b3e2d10","name":"Design REST API","description":"","priority":"HIGH","status":"IN_PROGRESS","dueDate":"2026-10-20","createdAt":"2026-10-07T09:50:00.000Z","updatedAt":"2026-10-07T10:10:00.000Z","project":{"id":"8f14e45f-ceeda-467a-9575-1c1a5b3e2d10","name":"Website Redesign"}}]}
```

API-11 · GET /api/tasks/{id} — *Purpose:* task details. *Success:* 200 { `data`: Task }. *Errors:* 400, 401, 404 `TASK_NOT_FOUND` (foreign and missing identical). *Security:* join to `project.ownerId`.

```
GET /api/tasks/c4ca4238-a0b9-4f23-8e51-9d1b2f6a7e33
```

```
200 OK
{"data":{"id":"c4ca4238-a0b9-4f23-8e51-9d1b2f6a7e33","projectId":"8f14e45f-ceeda-467a-9575-1c1a5b3e2d10","name":"Design REST API","description":"","priority":"HIGH","status":"IN_PROGRESS","dueDate":"2026-10-20","createdAt":"2026-10-07T09:50:00.000Z","updatedAt":"2026-10-07T10:10:00.000Z","project":{"id":"8f14e45f-ceeda-467a-9575-1c1a5b3e2d10","name":"Website Redesign"}}}
```

API-12 · POST /api/tasks — *Purpose:* create a task in an owned project. *Body:*

`projectId`, `name` (required); `description`, `priority`, `status`, `dueDate`. *Validation:* TASK-001..006. *Success:* 201 { `data`: Task }. *Errors:* 400; 401; 404 `PROJECT_NOT_FOUND` (foreign or missing project; nothing created). *Security:* project ownership verified before insert.

```
POST /api/tasks
{"projectId":"8f14e45f-ceed-467a-9575-1c1a5b3e2d10","name":"Design REST API","priority":"HIGH","dueDate":"2026-10-20"}
```

201 Created

```
{"data":{"id":"c4ca4238-a0b9-4f23-8e51-9d1b2f6a7e33","projectId":"8f14e45f-ceed-467a-9575-1c1a5b3e2d10","name":"Design REST API","description":"","priority":"HIGH","status":"PENDING","dueDate":"2026-10-20","createdAt":"2026-10-07T09:50:00.000Z","updatedAt":"2026-10-07T09:50:00.000Z","project":{"id":"8f14e45f-ceed-467a-9575-1c1a5b3e2d10","name":"Website Redesign"}}}
```

API-13 · PUT /api/tasks/{id} — *Purpose:* update a task, including marking it completed and changing status/priority. *Body:* any subset of name, description, priority, status, dueDate ; ≥ 1 field; projectId forbidden. *Success:* 200 { data: Task } (idempotent for repeated identical updates). *Errors:* 400, 401, 404 TASK_NOT_FOUND . *Security:* ownership via project in the same query as the update.

```
PUT /api/tasks/c4ca4238-a0b9-4f23-8e51-9d1b2f6a7e33
{"status":"COMPLETED"}
```

200 OK

```
{"data":{"id":"c4ca4238-a0b9-4f23-8e51-9d1b2f6a7e33","projectId":"8f14e45f-ceed-467a-9575-1c1a5b3e2d10","name":"Design REST API","description":"","priority":"HIGH","status":"COMPLETED","dueDate":"2026-10-20","createdAt":"2026-10-07T09:50:00.000Z","updatedAt":"2026-10-07T11:00:00.000Z","project":{"id":"8f14e45f-ceed-467a-9575-1c1a5b3e2d10","name":"Website Redesign"}}}
```

API-14 · DELETE /api/tasks/{id} — *Purpose:* delete a task. *Success:* 204. *Errors:* 400, 401, 404 TASK_NOT_FOUND . *Security:* ownership via project.

```
DELETE /api/tasks/c4ca4238-a0b9-4f23-8e51-9d1b2f6a7e33
```

204 No Content

18.5 Dashboard endpoint

API-15 · GET /api/dashboard — *Purpose:* statistics for the authenticated user. *Auth:* required. *Params:* none. *Success:* 200 { data: { totalProjects, totalTasks, completedTasks, pendingTasks, projectsInProgress, inProgressTasks } } (the last field is R; first five are M). *Errors:* 401, 500/503. *Security:* user id from JWT only; no way to request another user's stats.

```
GET /api/dashboard
```

```
200 OK
```

```
{"data":
```

```
{"totalProjects":3,"totalTasks":12,"completedTasks":5,"pendingTasks":4,"projectsInProgress":2,"inProgressTasks":3}}
```

18.6 Additional recommended endpoints (not part of the PDF minimum)

GET /api/health (public; { data: { status: "ok" } } ; 503 if DB unreachable) and GET /api/docs (Swagger UI backed by /docs/openapi.yaml). Both are R. No other endpoints may be added to the mandatory scope.

19. HTTP Status Code Standard

Code	Use when
200 OK	Successful GET, login, PUT (body returned)
201 Created	Successful POST that creates a resource (register, project, task)
204 No Content	Successful DELETE and logout (no body)
400 Bad Request	Validation failure (body/query/path), malformed JSON, unknown fields, malformed UUID, invalid dates/enums, empty strings
401 Unauthorized	Missing, malformed, invalid, or expired token; failed login
403 Forbidden	Reserved for RBAC (bonus). Not used in v1: cross-user access returns 404 to avoid leaking existence (OD-11)
404 Not Found	Resource missing OR not owned by the caller; unknown route
409 Conflict	Duplicate email on registration (including unique-constraint race)
413 Payload Too Large	Body larger than the 100 kb limit
422 Unprocessable Entity	Not used (OD-10): all validation failures use 400 consistently

Code	Use when
429 Too Many Requests	Auth rate limit exceeded; includes <code>Retry-After</code>
500 Internal Server Error	Unexpected error; generic message, details only in server logs
503 Service Unavailable	Database unreachable (R)

20. Authentication Architecture

JWT is mandatory (M); the details are recommended (R).

```
sequenceDiagram
    participant C as Client (web or Android)
    participant A as Express API
    participant D as PostgreSQL
    C->>A: POST /api/auth/register (fullName, email, password)
    A->>A: validate, normalize email, bcrypt hash cost 12
    A->>D: INSERT user (email UNIQUE)
    A-->>C: 201 user + JWT
    C->>C: store token (web localStorage, Android SecureStore)
    C->>A: GET /api/projects with Authorization Bearer token
    A->>A: verify signature + exp, read sub as user id
    A->>D: SELECT projects WHERE owner_id = sub
    A-->>C: 200 data
    Note over C,A: token expires
    C->>A: any request with expired token
    A-->>C: 401 TOKEN_EXPIRED
    C->>C: clear token + cache, show Login with session-expired message
```

Stage	Behavior
Registration	Validate → normalize email → bcrypt hash → insert (unique email) → issue JWT
Login	Validate → find user by normalized email → bcrypt compare (dummy hash if unknown) → issue JWT → client stores it securely
Authenticated request	Authorization: Bearer <JWT> → authenticate middleware verifies signature (algorithms pinned to <code>HS256</code> ; <code>none</code> rejected), checks <code>exp</code> , loads user by <code>sub</code> , sets <code>req.userId</code> → validation → controller → service applies ownership

Stage	Behavior
Expired token	401 <code>TOKEN_EXPIRED</code> → client clears token and caches → Login with <code>Your session has expired. Please log in again.</code>
Logout	Client deletes token and cached data; <code>POST /api/auth/logout</code> returns 204 (no revocation, OD-03)

- JWT: `HS256`; claims `sub`, `iat`, `exp`; secret ≥ 32 random bytes from `JWT_SECRET`; lifetime from `JWT_EXPIRES_IN` (default `1d`, OD-05). Tokens travel only in the `Authorization` header (no cookies, hence no CSRF surface).
- Never expose password hashes, the secret, or token internals in any response or log.
- Refresh tokens are OPTIONAL/BONUS (§37); without them users re-login after expiry.

21. Authorization Model

Ownership-based (M). Every project has one `ownerId`; every task belongs to a project, so **task ownership is resolved through its project**. The caller id always comes from the verified JWT, never from client input.

Operation	Rule	Enforcement (data layer)	Failure
List projects	Only own	<code>where: { ownerId: caller }</code>	— (empty list)
Read project	Owner only	<code>where: { id, ownerId: caller }</code>	404 <code>PROJECT_NOT_FOUND</code>
Update project	Owner only	<code>updateMany where { id, ownerId }</code> (count 0 → 404)	404
Delete project	Owner only	<code>deleteMany where { id, ownerId }</code>	404
List tasks	Own projects' tasks	<code>where: { project: { ownerId: caller } }</code> (+ <code>projectId</code> check)	404 if <code>projectId</code> foreign
Read task	Owner of its project	<code>where: { id, project: { ownerId: caller } }</code>	404 <code>TASK_NOT_FOUND</code>

Operation	Rule	Enforcement (data layer)	Failure
Create task	Owner of target project	Look up <code>project</code> where <code>{ id: body.projectId, ownerId: caller }</code> first	404 <code>PROJECT_NOT_FOUND</code>
Update task	Owner of its project; <code>projectId</code> immutable	<code>updateMany</code> where <code>{ id, project: { ownerId } }</code>	404
Delete task	Owner of its project	<code>deleteMany</code> where <code>{ id, project: { ownerId } }</code>	404
Dashboard	Own data only	All counts filtered by <code>ownerId = caller / project.ownerId = caller</code>	—

IDOR/BOLA considerations (M intent, R mechanics):

- UUIDs reduce guessing but are not authorization; the predicate is mandatory on every query.
- Foreign and missing resources return an identical 404 body and similar timing; counts or error text never reveal another user's data.
- `userId / ownerId` in a body or query is rejected (strict schemas); `projectId` in bodies/queries is re-verified on every request.
- Ownership is enforced in the service/data layer so a future route cannot bypass it by forgetting a controller check.
- A two-user test matrix covers every one of the 10 resource endpoints (AC-SEC-03).

22. Security Requirements

ID	Requirement	Tag
SEC-001	Passwords hashed with bcrypt (cost ≥ 12).	M
SEC-002	Plain-text passwords never stored, logged or returned.	M
SEC-003	JWT authentication on protected APIs.	M

ID	Requirement	Tag
SEC-004	Authentication middleware and protected routes: all endpoints except register, login, logout, health and docs require a valid token; new routes are protected by default. Web <code>ProtectedRoute</code> and mobile auth gating mirror this.	M
SEC-005	Authorization with ownership checks for view/modify/delete of projects and tasks on web and mobile (§21).	M
SEC-006	Backend validates every incoming request regardless of client: required fields, email format, date validity, empty strings, enum values, UUIDs, lengths; returns 400 with field details.	M
SEC-007	Sensitive information is never exposed in API responses (no <code>passwordHash</code> , secrets, stack traces, internal ids beyond the documented shapes).	M
SEC-008	SQL-injection protection through Prisma/parameterized queries; no raw user input in SQL (DBS-002).	M
SEC-009	Rate limiting on authentication endpoints (AUTH-013).	M
SEC-010	CORS configured for the web app's domain: allowlist from <code>CORS_ORIGIN</code> (comma-separated exact origins); methods <code>GET</code> , <code>POST</code> , <code>PUT</code> , <code>DELETE</code> , <code>OPTIONS</code> ; allowed headers <code>Authorization</code> , <code>Content-Type</code> ; credentials off; no <code>*</code> in production. Native Android requests carry no Origin and need no CORS entry.	M / R
SEC-011	Secrets only in environment variables; <code>.env</code> ignored by Git; <code>.env.example</code> documents every variable with placeholder values. Env validated at boot.	M (env var docs) / R
SEC-012	Safe error messages: generic text for 401 credentials and 500; no stack traces, SQL, or file paths in responses.	R
SEC-013	Mobile token stored only in secure storage (MOB-010).	M

ID	Requirement	Tag
SEC-014	Token expiration handled end-to-end (AUTH-017, MOB-011, WEB-006).	M
SEC-015	<code>helmet</code> default headers; <code>X-Powered-By</code> disabled.	R
SEC-016	HTTPS only in deployment (platform TLS), HSTS on the web host; mobile release builds reject cleartext.	R
SEC-017	Dependency hygiene: lockfiles committed; <code>npm audit</code> reviewed before release; automated updates optional.	R
SEC-018	Request size limit 100 kb (413 beyond); string length caps enforced by schemas.	R
SEC-019	Input handling: strict Zod schemas (trim, length, enum); output encoding by React/React Native (no raw HTML rendering); rich text not supported.	R
SEC-020	Production logging with redaction (<code>authorization</code> , <code>password</code> , <code>passwordHash</code> , <code>token</code>); secret-rotation procedure documented (rotating <code>JWT_SECRET</code> invalidates all sessions).	R
SEC-021	Mass-assignment prevention via strict schemas and explicit field mapping in services.	R
SEC-022	Secret scanning (e.g. gitleaks) before publishing the public repository; commit history reviewed for leaked secrets.	R

23. Error Handling

Layer	Requirements
Backend (M)	Central <code>errorHandler</code> returns the §18.1 envelope for every failure; domain errors map to 400/404/409; JWT errors → 401 codes; Prisma <code>P2002</code> → 409, <code>P2025</code> → 404; unknown errors → 500 <code>INTERNAL_ERROR</code> with generic message; stack traces only in non-production logs; every error logged server-side with <code>requestId</code> ; malformed JSON → 400; async handler wrapper so no unhandled rejections
Web (M)	Errors normalized to <code>AppError</code> ; user-friendly text (never raw codes/stack); Retry actions on failed loads; field errors inline; page-level <code>role=alert</code> messages; Error Boundaries; no blank screens; mutation failure keeps the form open with input intact
Mobile (M)	Network error/timeout (15 s) → offline message + Retry; 401 expiry → Login with message; pull-to-refresh failure → snackbar and previous data kept; mutation failure keeps form values; error boundary fallback screen; no crashes on expected failures

24. Loading / Empty / Error States

Screen	Loading	Empty	Success	Error
Login / Register	Button spinner, inputs disabled	n/a	Redirect to Dashboard	Inline field errors; alert for <code>Invalid email or password.</code> , <code>An account with this email already exists.</code> , <code>Too many attempts.</code> Please try again later.

Screen	Loading	Empty	Success	Error
Dashboard	Skeleton stat cards	Zeros + Nothing here yet. Create your first project to get started.	Five stat cards	Unable to load your dashboard. + Retry
Project List	Skeleton rows/cards	You don't have any projects yet. (+ Create project); filtered: No projects match your search or filters.	List	Unable to load projects. + Retry
Project Details	Skeleton header + rows	Tasks empty: This project has no tasks yet.	Details + tasks	Unable to load this project. + Retry; 404: Project not found. + back link
Project form	Submit spinner	n/a	Toast Project created. / Project updated.	Inline field errors; generic alert
Tasks (global)	Skeleton rows	You don't have any tasks yet.; filtered: No tasks match your search or filters.	List	Unable to load tasks. + Retry
Task form / quick actions	Submit/control spinner	n/a	Toast Task created. / Task updated. / Task deleted.	Inline errors; on failure control reverts + toast
Profile	Skeleton	n/a	Name, email	Retry

Screen	Loading	Empty	Success	Error
Global	—	—	—	Network: Unable to connect. Please check your internet connection and try again. Expired token: Your session has expired. Please log in again. Server: Something went wrong. Please try again.

25. Non-Functional Requirements

ID	Area	Requirement	Tag
NFR-001	Performance	List and dashboard endpoints respond with $p95 \leq 500$ ms on a seed of 50 projects / 1,000 tasks (excluding free-tier cold starts); indexes per §16; no N+1 queries (aggregate counts)	R
NFR-002	Performance	Avoid unnecessary requests: debounce search, cache per query key, cancel stale requests; web initial JS $\leq \sim 300$ KB gzipped	R
NFR-003	Reliability	Expected conditions (offline, expiry, validation, not-found) never crash or blank-screen any client; <code>/api/health</code> for monitoring	M (no crash/blank) / R
NFR-004	Maintainability	Modular architecture, reusable components, clear naming, TypeScript strict, ESLint + Prettier, no business logic in routes or JSX	M (clean structure) / R

ID	Area	Requirement	Tag
NFR-005	Scalability	Stateless API (horizontally scalable); DB indexes; pagination ready (bonus); rate-limit store swappable for a shared store	R
NFR-006	Accessibility	Keyboard accessible web, labeled inputs, AA contrast, accessible forms and dialogs (UX-016)	R
NFR-007	Compatibility	Modern browsers (WEB-012); Android 8.0+ (MOB requirement chosen in R)	M (Android) / R
NFR-008	Security	As §22	M
NFR-009	Observability	Structured logs with request ids; errors visible in the host's log viewer	M (logging) / R
NFR-010	Portability	Runs locally with documented commands; Docker Compose is BONUS	M (easy to run) / B

26. Testing Strategy

The PDF lists Unit Tests and Integration Tests as **OPTIONAL/BONUS (B)**. This PRD still defines the strategy; **tests that prove mandatory security and ownership behavior (the ·PO· set below) are required verification of mandatory requirements (R)** and may be automated or, for mobile device checks, executed as a documented manual script. Test cases share suffixes with acceptance criteria (§27): AC-X-nn is verified by TC-X-nn.

Level	Scope	Tooling (R)
Unit	Zod schemas (validation), auth service (hash/verify/JWT), ownership helpers, date rules, search-term escaping, error mapper, UI utilities	Vitest or Jest
Integration / API	Every endpoint against a real PostgreSQL test database via the Express app	Jest/Vitest + Supertest; test DB (Docker or CI service); <code>prisma migrate deploy</code> before tests; truncate tables between tests

Level	Scope	Tooling (R)
Security	Cross-user access, SQL-injection strings, rate limiting, invalid/expired/malformed JWT, mass assignment, response redaction	Same harness
Web	Protected routes, forms, search/filter, dashboard, error states	Vitest + React Testing Library + MSW; Playwright E2E smoke (Could)
Mobile	Login/logout, secure storage mocked, pull-to-refresh, no network, expired token	Jest (<code>jest-expo</code>) + React Native Testing Library; manual script on a physical/emulated Android device
Cross-platform	S1-S3 sync scenarios run against the deployed or local backend	Manual script with checklist; optional API-level automation

Test catalog by category (details in §27):

Category	Test cases
Authentication unit/integration	TC-AUTH-01..09 ·P0·
Projects	TC-PROJ-01..09
Tasks	TC-TASK-01..10
Dashboard	TC-DASH-01..04
Search/filter	TC-SRCH-01..06 ·P0· (05 is the injection test)
Security	TC-SEC-01..12 ·P0· (03 cross-user matrix, 06 SQL injection, 07 rate limit)
Mobile	TC-MOB-01..10 (08 secure storage, 09 expired token, 10 offline)
Cross-platform	TC-SYNC-01..03
Docs / deployment / submission	TC-DOC-01..03, TC-DEP-01..04, TC-SUB-01..02 (checklist-style)

Rules: tests are independent and deterministic; test data is synthetic; test DB is never the production DB; CI (bonus) runs lint, type-check and tests on every push.

27. Acceptance Criteria Matrix

Every mandatory requirement has at least one objective criterion. Each AC is verified by the TC with the same suffix. Priority: Must (blocks completion) / Should.

Req ID	AC ID	Acceptance criterion (measurable)	Test method	Pri
AUTH-001..008	AC-AUTH-01	POST /register with valid data → 201; body has user (id, fullName, email, createdAt) and token, no passwordHash; DB row holds a bcrypt hash (\$2a\$ / \$2b\$, cost ≥ 12), never the plaintext	API test + DB assertion	Must
AUTH-004, 007	AC-AUTH-02	Same email again (also different case/surrounding spaces) → 409 EMAIL_ALREADY_REGISTERED; exactly one users row; two concurrent registrations yield one 201 and one 409	API test	Must
AUTH-001..005	AC-AUTH-03	Missing field, invalid email, blank name, empty password, 7-char password, > 72-byte password → 400 with per-field details; no row created	API + unit	Must
AUTH-009..012	AC-AUTH-04	Valid login → 200 + JWT (HS256, sub = user id, exp = iat + configured lifetime); wrong password and unknown email → 401 with identical body	API test	Must
AUTH-013	AC-AUTH-05	11th failed login from one IP inside 15 min → 429 RATE_LIMITED with Retry-After; successful logins not counted; window resets	API test (small window via env)	Must
AUTH-015, 016	AC-AUTH-06	GET /me with valid token returns exactly { id, fullName, email, createdAt }; without token 401 UNAUTHENTICATED; app restart with valid token remains signed in	API + client test	Must

Req ID	AC ID	Acceptance criterion (measurable)	Test method	Pri
AUTH-017, 018	AC-AUTH-07	Expired, malformed, wrong-signature, <code>alg:none</code> and deleted-user tokens → 401 (<code>TOKEN_EXPIRED</code> / <code>TOKEN_INVALID</code>), no data in body; clients show Login (with expiry message for expired)	API + client test	Must
AUTH-014	AC-AUTH-08	<code>POST /logout</code> → 204 with valid, expired and missing token; web and mobile delete stored token + cached data and show Login; protected URLs then redirect to Login	API + client test	Must
AUTH-020	AC-AUTH-09	Account registered on web logs in on Android and vice versa; <code>/me</code> returns the same <code>user.id</code>	Manual cross-platform	Must
PROJ-001, 002	AC-PROJ-01	Create → 201; <code>ownerId</code> = caller; <code>status</code> defaults <code>NOT_STARTED</code> ; <code>createdAt</code> server-set; body <code>id</code> / <code>ownerId</code> → 400	API test	Must
PROJ-001, 005	AC-PROJ-02	Missing, empty, whitespace or > 120-char name → 400; date sent as <code>" "</code> → 400	API test	Must
PROJ-003, 004	AC-PROJ-03	Invalid status, impossible date (<code>2026-02-30</code>), malformed date, <code>endDate</code> < <code>startDate</code> (also on update using merged values) → 400; equal dates accepted	API + unit	Must
PROJ-006	AC-PROJ-04	With users A and B each owning projects, <code>GET /projects</code> returns only the caller's; B's ids never appear for A	API test	Must
PROJ-008..011	AC-PROJ-05	GET/PUT/DELETE on another user's project id → 404 <code>PROJECT_NOT_FOUND</code> , identical to a nonexistent id; database unchanged	API cross-user test	Must

Req ID	AC ID	Acceptance criterion (measurable)	Test method	Pri
PROJ-009	AC-PROJ-06	PUT subset updates only those fields; {} → 400; ownerId / id / createdAt in body → 400	API test	Must
PROJ-010	AC-PROJ-07	DELETE own project having 3 tasks → 204; project and its 3 tasks are gone; subsequent GET → 404	API + DB test	Must
PROJ-014, 016	AC-PROJ-08	Web and mobile show loading, empty (You don't have any projects yet.), error + Retry, status badges with text, and a delete confirmation naming the project and its task count	Component test + manual	Must
PROJ-007	AC-PROJ-09	taskCount and completedTaskCount in list and detail equal DB counts	API test	Should
TASK-001, 002	AC-TASK-01	POST /tasks in own project → 201; defaults MEDIUM / PENDING ; includes project {id,name}	API test	Must
TASK-003	AC-TASK-02	projectId of another user's or nonexistent project → 404 PROJECT_NOT_FOUND ; no row created	API cross-user test	Must
TASK-004..006	AC-TASK-03	Blank name, invalid priority/status, impossible or malformed dueDate , "" dueDate → 400; null dueDate accepted	API test	Must
TASK-007, 008, 010	AC-TASK-04	GET /tasks returns only tasks of caller's projects; GET /tasks/{id} of foreign task → 404 TASK_NOT_FOUND	API cross-user test	Must
TASK-011	AC-TASK-05	PUT on foreign task → 404 and no change; PUT containing projectId → 400; subset update persists	API test	Must
TASK-014	AC-TASK-06	DELETE own task → 204 then GET → 404; DELETE foreign task → 404 and row intact	API test	Must

Req ID	AC ID	Acceptance criterion (measurable)	Test method	Pri
TASK-012	AC-TASK-07	PUT {status:COMPLETED} sets COMPLETED ; repeating returns 200 unchanged; dashboard completedTasks increments exactly once	API test	Must
TASK-013	AC-TASK-08	Status and priority changed via quick actions on web and Android persist and show on the other platform after refresh	Manual + API	Must
TASK-009, 019	AC-TASK-09	GET /tasks?projectId= returns only that project's tasks; empty project shows This project has no tasks yet. ; foreign projectId → 404	API + component	Must
TASK-016, 019	AC-TASK-10	Task screens show loading, empty, error + Retry, delete confirmation; submit disabled while pending (double click creates one task); Overdue label rule applied	Component + manual	Must
DASH-001	AC-DASH-01	Response contains numeric totalProjects , totalTasks , completedTasks , pendingTasks , projectsInProgress	API test	Must
DASH-002, 004	AC-DASH-02	With two seeded users each user's values equal DB counts for own data only; totalTasks = pending + inProgress + completed	API test	Must
DASH-007, 008	AC-DASH-03	New user sees five zeros + hint; failure shows alert + Retry on web and mobile	Component + manual	Must
DASH-006	AC-DASH-04	After create/complete/delete the dashboard shows updated values on next view (web) / pull-to-refresh (mobile)	Manual + component	Must
SRCH-001, 003	AC-SRCH-01	search=web matches Website , my WEB app , web (case-insensitive substring) and excludes others	API test	Must

Req ID	AC ID	Acceptance criterion (measurable)	Test method	Pri
SRCH-001	AC-SRCH-02	Project <code>status</code> filter returns only that status	API test	Must
SRCH-002, 005	AC-SRCH-03	Task <code>search</code> , <code>status</code> , <code>priority</code> , <code>projectId</code> work alone and combined with AND, returning exactly the expected set	API test	Must
SRCH-004, 006	AC-SRCH-04	Invalid status/priority → 400; search > 100 chars → 400; repeated parameter → 400; whitespace-only search ignored	API test	Must
SRCH-008	AC-SRCH-05	Search strings <code>' OR '1'='1 , % , _ , \ , " ; DROP TABLE tasks ; --</code> are literal: no error, no extra rows, tables intact; code scan finds no <code>\$queryRawUnsafe</code> or string-built SQL	API test + review	Must
SRCH-009, 010	AC-SRCH-06	No match → filtered-empty message with Clear filters; clearing restores the full list; web search/filter state survives reload (URL)	Component + manual	Must
MOB-001	AC-MOB-01	Release APK installs and launches on Android 8+ device/emulator	Manual	Must
MOB-003	AC-MOB-02	Register, login, logout on Android against the deployed backend with the account used on web	Manual	Must
MOB-004	AC-MOB-03	Android dashboard shows the five metrics equal to web for the same account	Manual	Must
MOB-005	AC-MOB-04	Android lists own projects; opening a project shows its tasks	Manual	Must
MOB-006, 007	AC-MOB-05	Create, edit, delete (with confirmation), complete, change status, change priority on Android; verified on server	Manual + API	Must

Req ID	AC ID	Acceptance criterion (measurable)	Test method	Pri
MOB-008	AC-MOB-06	Task search by name and filters by status and priority on Android send server parameters and show correct results	Manual + network log	Must
MOB-009	AC-MOB-07	Pull-to-refresh issues a new API request and shows a change made on web	Manual	Must
MOB-010	AC-MOB-08	Token exists only in SecureStore (no AsyncStorage/MMKV/file storage of the token in code); removed on logout	Code review + manual	Must
MOB-011	AC-MOB-09	Expired token → Login screen with <code>Your session has expired. Please log in again.</code> ; stored token cleared	Manual (short-lived token)	Must
MOB-012	AC-MOB-10	Airplane mode on launch/refresh/submit → <code>Unable to connect. Please check your internet connection and try again.</code> + Retry; no crash or blank screen; form input kept	Manual	Must
SYNC-002	AC-SYNC-01	Task created on web appears on Android after pull-to-refresh with identical fields	Manual	Must
SYNC-002	AC-SYNC-02	Status changed on Android appears on web after refresh; dashboard counts agree	Manual	Must
SYNC-002	AC-SYNC-03	Task deleted on web disappears from Android after pull-to-refresh	Manual	Must
SEC-001, 002	AC-SEC-01	Database scan shows no plaintext passwords (all <code>password_hash</code> bcrypt); logs contain no passwords	DB query + log grep	Must
SEC-003, 004	AC-SEC-02	Route enumeration: every non-public route returns 401 without a token	API test	Must

Req ID	AC ID	Acceptance criterion (measurable)	Test method	Pri
SEC-005	AC-SEC-03	Two-user matrix over all 10 resource endpoints (plus body/query <code>projectId</code>): every foreign id → 404, zero side effects	API test	Must
SEC-006	AC-SEC-04	Invalid-payload table (missing, blank, email, date, enum, UUID, extra field) → 400 on every write endpoint	API test	Must
SEC-007	AC-SEC-05	No response contains <code>passwordHash</code> , secrets or stack traces (success and error paths, <code>NODE_ENV=production</code>)	API test	Must
SEC-008	AC-SEC-06	Injection strings in search, ids and body fields change nothing; no raw SQL concatenation (review + lint rule)	API test + review	Must
SEC-009	AC-SEC-07	Login limit (AC-AUTH-05) and register limit (10/hour/IP) enforced	API test	Must
SEC-010	AC-SEC-08	Origin = deployed web domain receives <code>Access-Control-Allow-Origin</code> ; any other origin does not; preflight succeeds	API test + browser check	Must
SEC-011, 022	AC-SEC-09	Secret scan clean; <code>.env</code> ignored; only <code>.env.example</code> committed; server refuses to start without a valid <code>JWT_SECRET</code>	Scan + boot test	Must
SEC-015, 018	AC-SEC-10	Helmet headers present; <code>X-Powered-By</code> absent; 101 kb body → 413	API test	Should
SEC-021	AC-SEC-11	Body containing <code>userId</code> , <code>ownerId</code> , <code>id</code> , <code>createdAt</code> → 400	API test	Must
SEC-012, 020	AC-SEC-12	In production mode 500 responses are generic; logs redact <code>Authorization</code> and passwords	API test + log check	Must

Req ID	AC ID	Acceptance criterion (measurable)	Test method	Pri
API-001	AC-API-01	All 15 endpoints exist with the specified methods/paths and appear in the OpenAPI file	API test + doc review	Must
API-001	AC-API-02	Web and mobile source reference the same <code>/api</code> paths and no other backend host	Code grep	Must
§18.1	AC-API-03	Every error response uses the standard envelope with <code>code</code> , <code>message</code> , <code>requestId</code>	API test	Must
NFR-001	AC-NFR-01	Indexes of §16 exist; p95 ≤ 500 ms on the seeded dataset (warm instance)	Query plan + load check	Should
DOC-001..005	AC-DOC-01	A fresh clone follows the README to run backend, web and mobile successfully (incl. mobile against the deployed API)	Walk-through	Must
DOC-004	AC-DOC-02	API documentation (OpenAPI served at <code>/api/docs</code> and <code>docs/openapi.yaml</code>) covers all endpoints with examples	Doc review	Must
DOC-006	AC-DOC-03	ER diagram and schema table exist in <code>docs/</code> and the README	Doc review	Must
DEP-001..004	AC-DEP-01	Backend reachable at public HTTPS URL; <code>/api/health</code> returns 200	HTTP check	Must
DEP-002	AC-DEP-02	Web URL loads; register/login work from the browser (CORS correct)	Manual	Must
DEP-003	AC-DEP-03	APK or distribution link installs and talks to the deployed API	Manual	Must
DEP-004	AC-DEP-04	Migrations applied to the deployed database; no secrets in the repo	Check	Must
SUB-001..008	AC-SUB-01	Repository is public and viewable in a logged-out/incognito browser	Manual	Must

Req ID	AC ID	Acceptance criterion (measurable)	Test method	Pri
SUB-007	AC-SUB-02	Recording ≤ 5 min shows: same-account login on web and mobile, task created on one, shown on the other	Review	Must

28. Edge Cases

Area	Edge case	Expected behavior
Auth	Duplicate email (any case/whitespace)	409; one row
Auth	Invalid email / blank name / empty password	400 with field details; inline messages
Auth	Wrong credentials; unknown email	Identical 401 Invalid email or password.
Auth	Expired JWT; malformed JWT	401 TOKEN_EXPIRED / TOKEN_INVALID ; session cleared; Login
Auth	Logout when already logged out	204; client simply lands on Login
Auth	Password > 72 bytes; password with unicode/spaces	Rejected (> 72) / accepted unchanged
Auth	Two browser tabs; one logs out	Other tab's next request → 401 → Login
Auth	Valid token for a deleted user	401 TOKEN_INVALID
Projects	Empty/whitespace name; invalid status	400
Projects	Invalid or impossible dates; end before start	400; equal dates accepted; null clears
Projects	Nonexistent or foreign id	404 PROJECT_NOT_FOUND (identical)
Projects	Delete project containing tasks	Confirmation shows count; cascade (§29)
Projects	Duplicate project names	Allowed (names are not unique)
Tasks	Empty name; invalid priority/status/dueDate	400

Area	Edge case	Expected behavior
Tasks	Nonexistent or foreign project in body/query	404 <code>PROJECT_NOT_FOUND</code> ; nothing created
Tasks	Foreign task id	404 <code>TASK_NOT_FOUND</code>
Tasks	Complete an already-completed task	200, unchanged, no error
Tasks	Past due date	Allowed; shows <code>Overdue</code> unless completed
Tasks	Project deleted (other device) while editing its task	404 → message <code>This task no longer exists.</code> + list refresh
Network	Offline	Offline message + Retry; no crash
Network	Timeout (15 s)	Treated as offline
Network	Server unavailable (502/503)	<code>Something went wrong. Please try again.</code> / offline message; Retry
Network	Several parallel requests fail with 401	Session cleared once; single redirect
UI	Empty lists	Empty-state copy (§24)
UI	120/150-char names; 2000-char descriptions	Truncate in lists, wrap in details; layout intact
UI	Slow API	Loading indicator; no blank screen
UI	Double click / double tap on submit	Button disabled while pending; one request
UI	Rapid filter changes	Stale responses ignored; final state matches last input
UI	Back button after logout	Login shown; no cached protected data (query cache cleared)

29. Project Delete / Task Relationship

DESIGN DECISION (RECOMMENDED ENGINEERING DECISION; PDF is silent — OD-02): deleting a project **cascade-deletes its tasks** in one transaction (`ON DELETE CASCADE`), preceded by an explicit confirmation that names the project and its task count.

Option	Pros	Cons	Verdict
Cascade delete (chosen)	One clear action; no orphan or blocked state; matches users' mental model ("delete this project"); simple, atomic	Irreversible loss of tasks	Chosen , mitigated by confirmation with task count
Restrict until tasks are removed (409)	Prevents accidental loss	Tedious for projects with many tasks; extra error path on web and mobile	Rejected
Soft delete / archive	Recoverable	Adds fields, filters and states beyond the PDF; larger scope	Rejected (Could, future)

Implications: API-09 returns 204 and the dashboard/task lists no longer include the deleted tasks; mobile Project Details must handle a project deleted on web (404 → message + back); test AC-PROJ-07.

30. Search and Filter API Design

Endpoint	Param	Type / values	Behavior
GET /api/projects	search	string, trimmed, ≤ 100	Case-insensitive substring on name
	status	NOT_STARTED IN_PROGRESS COMPLETED	Exact match
GET /api/tasks	search	string, trimmed, ≤ 100	Case-insensitive substring on name
	status	PENDING IN_PROGRESS COMPLETED	Exact match
	priority	LOW MEDIUM HIGH	Exact match
	projectId	UUID	Restrict to one owned project (404 if not owned)

Examples: `GET /api/projects?search=web`, `GET /api/projects?status=IN_PROGRESS`, `GET /api/tasks?search=api`, `GET /api/tasks?status=PENDING&priority=HIGH`, `GET /api/tasks?projectId=<uuid>&search=api&status=IN_PROGRESS&priority=HIGH`. Parameters are ANDed. Reserved for bonus: `sort`, `order`, `page`, `limit`.

Reference implementation (R) — all filtering through Prisma, owner predicate always present, values parameterized by the ORM:

```
const where: Prisma.TaskWhereInput = {
  project: { ownerId: userId },
  ...(projectId && { projectId }),
  ...(status && { status }),
  ...(priority && { priority }),
  ...(search && { name: { contains: search, mode: 'insensitive' } }),
};
```

The implementation MUST verify (test AC-SRCH-05) that `%`, `_` and `\` in `search` behave as literal characters; if the ORM does not escape them, escape before building the filter.

31. Deployment Architecture

The PDF requires deployment URLs for the web app and the backend and an Android APK or Expo/Firebase distribution link (**M**). Provider names below are **RECOMMENDED** examples; any equivalent platform is acceptable.

```
flowchart LR
    U1[Browser] -->|HTTPS| WEBHOST[Static host: React SPA build]
    WEBHOST -->|. serves JS bundle|. U1
    U1 -->|HTTPS REST + Bearer + CORS| API[Backend service: Node + Express]
    U2[Android phone - APK] -->|HTTPS REST + Bearer| API
    API -->|TLS via DATABASE_URL| DB[(Managed PostgreSQL)]
    REPO[Public Git repository] -->|build and deploy| WEBHOST
    REPO -->|deploy + prisma migrate deploy| API
    EAS[EAS Build or Firebase App Distribution] -->|APK or install link| U2
```

ID	Component	Requirement	Tag
DEP-001	Backend	Deployed as one Node service on a cloud platform (e.g. Render, Railway, Fly.io) with a public HTTPS URL. Start command runs the compiled server; build runs <code>prisma generate</code> . Health check <code>GET /api/health</code> . <code>TRUST_PROXY</code> set so rate limiting sees client IPs.	M (deployed) / R

ID	Component	Requirement	Tag
DEP-002	Web	Deployed as static build on a frontend host (e.g. Vercel, Netlify, Cloudflare Pages) with SPA fallback. <code>VITE_API_URL</code> points to the backend. The backend's <code>CORS_ORIGIN</code> contains exactly this web origin.	M (deployed) / R
DEP-003	Mobile	Android build via EAS Build (<code>preview</code> profile, <code>buildType: apk</code>) or Firebase App Distribution/Expo link. <code>EXPO_PUBLIC_API_BASE_URL</code> = deployed backend HTTPS URL baked at build time. Install instructions in README.	M (APK or distribution link) / R
DEP-004	Database	Managed PostgreSQL (e.g. Neon, Supabase, Render, Railway). Schema applied with <code>prisma migrate deploy</code> as a release step; connection over TLS; automatic backups on; app role least-privilege. Optional demo seed run once, manually.	M (PostgreSQL) / R
DEP-005	Operations	Free-tier services may sleep: warm the backend (open <code>/api/health</code>) before the demo and set the mobile/web request timeout and loading UI accordingly; record deployed URLs in README.	R

32. Environment Variables

Rule: anything prefixed `VITE_` or `EXPO_PUBLIC_` is embedded in client bundles and is PUBLIC. Backend secrets never appear in web or mobile code, builds or repos.

Variable	Component	Secret?	Example / default	Purpose
<code>DATABASE_URL</code>	Backend	Secret	<code>postgresql://user:pass@host:5432/pms?sslmode=require</code>	Prisma connection string
<code>JWT_SECRET</code>	Backend	Secret	<code>≥ 32 random bytes (e.g. <code>openssl rand -base64 48</code>)</code>	Signs/verifies JWTs
<code>JWT_EXPIRES_IN</code>	Backend	No	<code>1d</code>	Token lifetime (OD-05)
<code>PORT</code>	Backend	No	<code>4000</code>	HTTP port (platform may inject)

Variable	Component	Secret?	Example / default	Purpose
<code>CORS_ORIGIN</code>	Backend	No	<code>https://pms-web.example.app,http://localhost:5173</code>	Comma-separated allowed web origins
<code>NODE_ENV</code>	Backend	No	<code>development / test / production</code>	Mode (error detail, logging)
<code>BCRYPT_COST</code>	Backend	No	<code>12</code>	bcrypt work factor (R)
<code>LOG_LEVEL</code>	Backend	No	<code>info</code>	pino level (R)
<code>TRUST_PROXY</code>	Backend	No	<code>1</code> in production behind a proxy	Correct client IP for rate limiting (R)
<code>RATE_LIMIT_LOGIN_MAX / _WINDOW_MIN</code>	Backend	No	<code>10 / 15</code>	Failed-login limit (R)
<code>RATE_LIMIT_REGISTER_MAX / _WINDOW_MIN</code>	Backend	No	<code>10 / 60</code>	Register limit (R)
<code>TEST_DATABASE_URL</code>	Backend tests	Secret	separate database	Integration tests (R)
<code>VITE_API_URL</code>	Web	Public	<code>https://pms-api.example.app</code> (or <code>http://localhost:4000</code>)	Backend base URL (paths add <code>/api/...</code>)
<code>API_BASE_URL</code> → <code>EXPO_PUBLIC_API_BASE_URL</code>	Mobile	Public	<code>https://pms-api.example.app</code> ; emulator dev: <code>http://10.0.2.2:4000</code>	Backend base URL (Expo requires the <code>EXPO_PUBLIC_</code> prefix)

Deliver `backend/.env.example`, `web/.env.example`, `mobile/.env.example` with placeholders only; startup validation fails fast on missing/weak backend variables.

33. Documentation Requirements

ID	Requirement	Tag
DOC-001	Project setup instructions for backend, web and mobile (prerequisites, install, run, build)	M
DOC-002	Environment variable documentation (table as §32, public vs secret)	M
DOC-003	Database setup instructions (create database, <code>prisma migrate deploy / dev</code> , optional seed, reset)	M
DOC-004	API documentation: OpenAPI 3 file <code>docs/openapi.yaml</code> served by Swagger UI at <code>/api/docs</code> , covering all endpoints, schemas, errors, auth	M (API docs) / R (format)

ID	Requirement	Tag
DOC-005	How to run the mobile app against the deployed backend (set <code>EXPO_PUBLIC_API_BASE_URL</code> , run Expo or install the APK, test account)	M
DOC-006	Database schema / ER diagram (Mermaid in <code>docs/</code> + README, plus schema table)	M
DOC-007	README: overview, architecture diagram, tech stack, repo layout, quick start, deployed URLs, demo account (synthetic), testing instructions, security notes (token storage, rate limits, CORS), design decisions (ADRs summary), known limitations (stateless logout, last-write-wins), troubleshooting (CORS, emulator networking, cold start)	M (README) / R (content)

Documentation must let a developer who has never seen the project run everything. It is updated whenever behavior changes (Definition of Done).

34. Git / Repository Requirements

```

/backend      Express API, Prisma schema + migrations, tests
/web          React + Vite app
/mobile       Expo React Native app
/docs         openapi.yaml, er-diagram.md, architecture.md, demo-script.md
README.md
.gitignore

```

ID	Requirement	Tag
GIT-001	Public repository viewable without logging in; one monorepo (the PDF permits one repo or three)	M / R
GIT-002	<code>.gitignore</code> covers <code>node_modules</code> , <code>.env*</code> (except <code>.env.example</code>), build outputs (<code>dist</code> , <code>.expo</code> , <code>android/build</code>), logs, keystores (<code>*.jks</code> , <code>*.keystore</code>), OS/editor files	R
GIT-003	<code>.env.example</code> per package; no secrets, tokens, keystores or real personal data committed; secret scan before publishing	M (no secrets) / R

ID	Requirement	Tag
GIT-004	Meaningful, small commits (imperative messages, Conventional Commits style optional); feature work reviewed against this PRD; no large generated artifacts committed	R
GIT-005	Lockfiles committed; consistent Node version via <code>.nvmrc</code> / <code>engines</code>	R

35. Demo Requirements (5-minute screen recording)

PDF (M): log in with the same account on web and mobile, create a task on one, show it on the other. Preparation: deployed backend warmed; demo account (e.g.

`demo@example.com`, synthetic) with 2–3 projects and a few tasks; browser window and Android screen (phone mirror or emulator) both visible; no secrets on screen.

Time	Action	Evaluator should observe
0:00–0:30	State the architecture in one sentence; show web URL, backend <code>/api/health</code> , <code>/api/docs</code>	One backend + one database serve both apps
0:30–1:15	Web: log in with the demo account; show Dashboard (note the five numbers) and Projects	Login works; dashboard shows five metrics
1:15–2:00	Android: log in with the SAME account; show Dashboard and Projects	Identical user and numbers on both platforms
2:00–3:00	Web: open a project, create task <code>Demo task from web</code> (priority High); switch to Android, pull-to-refresh, open the project	New task appears on mobile — shared backend/database
3:00–4:00	Android: mark the task completed (and change priority); switch to web, refresh	Updated status/priority on web; Completed Tasks +1 on dashboard
4:00–4:30	Web: delete the task; Android pull-to-refresh	Task disappears on mobile

Time	Action	Evaluator should observe
4:30–5:00	Show search/filter once on mobile; mention security (ownership, bcrypt, JWT, secure storage); log out on both; show README/ER diagram links	Search/filter, logout, documentation present

Recording rules: ≤ 5:00; 1080p; URL bar visible; link shared with access for anyone with the link; rehearse once; keep a fallback second take.

36. Submission Checklist

ID	Item (all M)	Done when
SUB-001	Public GitHub repository link	Opens in a logged-out/incognito browser
SUB-002	Database schema / ER diagram	<code>docs/er-diagram.md</code> and README section
SUB-003	API documentation	<code>docs/openapi.yaml</code> + live <code>/api/docs</code> link
SUB-004	README	Covers DOC-001..007
SUB-005	Deployment URLs: web app and backend	Both live and working together
SUB-006	Android APK or Expo/Firebase distribution link	Installs and uses the deployed backend
SUB-007	5-minute screen recording	Matches §35
SUB-008	Test data only	No real personal data anywhere (repo, seed, demo)

Pre-submission verification: run the §45 quality gate; fresh-clone walk-through (AC-DOC-01); run all P0 tests; open every submitted link in a private window; confirm the APK on a clean device; confirm no secrets in Git history; confirm CORS from the deployed web origin; be ready to explain every design decision (PDF Important Notes).

37. Bonus Features (OPTIONAL / BONUS)

All items below are **B**: optional per the PDF, considered positively in evaluation, and forbidden from delaying or destabilizing mandatory work. Start only when every P0 item and the §45 gate are complete.

Bonus	Value	Effort	Notes and guardrails
Unit tests	High	Low–Med	Also supports §26; start with validation, auth, ownership
Integration tests	High	Med	The P0 security tests already form the core; extend to all endpoints
Docker support	High	Low	<code>docker-compose.yml</code> with PostgreSQL + backend (+ web optional); helps the "easy to run" requirement
CI/CD pipeline	Medium	Low–Med	GitHub Actions: install, lint, type-check, test (with Postgres service), build; optional deploy hooks
Pagination	Medium	Med	<code>page / limit</code> with <code>meta { total, page, limit }</code> ; defaults must keep current clients working; add UI paging/infinite scroll
Sorting	Medium	Low	<code>sort (createdAt , name , dueDate , priority , status) + order ;</code> whitelist fields (no raw column names)
Shared types / validation	Medium	Med	Shared package (e.g. <code>/shared</code>) exporting Zod schemas and types for backend, web and mobile
Refresh tokens	Medium	Med–High	Short-lived access token + rotating refresh token (hashed in DB, revocable); must keep AUTH-017 behavior and secure mobile storage
Audit logs	Low	Med	Append-only table for create/update/delete/login events; never log secrets
Role-based access control	Low	Med	Out of place for a single-user model; only if collaboration is added; ownership checks stay mandatory

Bonus	Value	Effort	Notes and guardrails
Push notifications for tasks due tomorrow	Low	High	expo-notifications , device-token storage, scheduled job; needs a development/EAS build, not Expo Go
Offline viewing of tasks on mobile	Low-Med	Med-High	Read-only cache of last fetched tasks (cleared on logout); backend remains source of truth; mutations stay online-only

Recommended order after P0: tests → Docker → CI/CD → sorting → pagination → shared validation → refresh tokens → offline viewing → audit logs → push notifications → RBAC.

38. Traceability Matrix

Every requirement in the source PDF has an SRC- row mapping: Assignment requirement → PRD requirement ID → backend → web → mobile → database impact → API → test case / acceptance criterion. Because TC-X-nn mirrors AC-X-nn (§26), the last column lists both at once. Antigravity MUST re-verify every row before declaring completion (§43, item 20).

38.1 Supplemental acceptance criteria (for mandatory technical requirements not covered in §27)

Req ID	AC ID	Acceptance criterion	Test method	Pri
WEB-001..013, UX-001	AC-WEB-01	At 320, 768 and 1280 px widths every web screen is usable with no horizontal page scroll	Manual + browser devtools	Must
WEB-008, UX-004, UX-009, UX-011, WEB-009	AC-WEB-02	Web forms show inline validation, submit buttons show a loading state, slow or failed requests show a loading indicator or error message with Retry, and no screen goes blank	Component tests + manual	Must

Req ID	AC ID	Acceptance criterion	Test method	Pri
WEB-002, WEB-003, WEB-006	AC-WEB-03	Code review: feature folders and reusable components exist; HTTP calls only through <code>lib/api</code> ; no component exceeds ~200 lines without justification	Code review	Must
MAPP-001..012, MOB-013..015	AC-MAPP-01	Mobile has auth stack + tab navigation, form validation, loading indicators, error handling, and the structure of MAPP-002; layouts are phone-first	Code review + manual	Must
§15, NFR-004, NFR-009	AC-BE-01	Code review/lint: per-resource route files, middleware chain, controller → service → data separation, centralized error handler, no business logic in routes; request logs show method, path, status, duration, requestId	Code review + log sample	Must
§16, DBS-003	AC-DB-01	Migrations create <code>users</code> , <code>projects</code> , <code>tasks</code> with PKs, FKs, unique email, enums and CHECKs; no derived columns (3NF); inserting a task with a nonexistent <code>project_id</code> fails	Migration test + DB query	Must

38.2 Matrix

SRC	Assignment requirement (PDF section)	PRD IDs	Backend	Web	Mobile	Database	API	AC / TC
SRC-001	Build a web app and a mobile app (Intro)	SC-07, SC-08	one Express app	/web React SPA	/mobile Expo app	—	all	AC-WEB-01, AC-MOB-01

SRC	Assignment requirement (PDF section)	PRD IDs	Backend	Web	Mobile	Database	API	AC / TC
SRC-002	Both apps use the same backend and database (Intro, §6)	SYNC-001, MOB-002, API-001	single deployable	VITE_API_URL	EXPO_PUBLIC_API_BASE_URL	one PostgreSQL	all 15	AC-API-02, AC-SYNC-01..03
SRC-003	A user logs in on either and sees/manages the same data (Intro)	AUTH-020, SYNC-002	JWT, owner scoping	same endpoints	same endpoints	users, projects, tasks	all	AC-AUTH-09, AC-SYNC-01..03
SRC-004	Create projects, organize tasks in projects, track progress, dashboard (Intro)	F-PROJ, F-TASK, F-DASH, PROJ-007	services	pages	screens	projects, tasks	projects, tasks, dashboard	AC-PROJ-01, AC-TASK-01, AC-DASH-01
SRC-010	User registration (§1)	AUTH-001..008	auth.service.register	RegisterPage	RegisterScreen	users	POST /auth/register	AC-AUTH-01..03
SRC-011	User login (§1)	AUTH-009..013	auth.service.login, authLimiter	LoginPage	LoginScreen	users	POST /auth/login	AC-AUTH-04, 05
SRC-012	User logout (§1)	AUTH-014	logout handler (204)	AuthProvider.logout	logout + SecureStore delete	—	POST /auth/logout	AC-AUTH-08
SRC-013	User fields: Full Name, Email Address, Password (§1)	AUTH-001..005	Zod auth schemas	forms	forms	full_name, email, password_hash	register	AC-AUTH-01, 03
SRC-014	Email addresses must be unique (§1)	AUTH-004, 007	check + P2002→409	inline error	inline error	users.email UNIQUE	register	AC-AUTH-02
SRC-015	Passwords never stored in plain text (§1)	AUTH-006, SEC-001/002, DBS-001	bcrypt	—	—	password_hash	register, login	AC-AUTH-01, AC-SEC-01
SRC-016	Stay logged in until logout or token expiration (§1)	AUTH-012, 016	JWT exp, /me	localStorage + /me	SecureStore + /me	—	GET /auth/me	AC-AUTH-06, 07
SRC-017	One account works on web and mobile (§1)	AUTH-020, SYNC-004	shared auth	same API	same API	users	auth endpoints	AC-AUTH-09
SRC-020	Create a project (§2)	PROJ-001..005	projects.service.create	ProjectForm	(Should) project form	projects	POST /projects	AC-PROJ-01..03
SRC-021	View project details (§2)	PROJ-008, 014	getById	ProjectDetailsPage	ProjectDetailsScreen	projects	GET /projects/{id}	AC-PROJ-05, 08
SRC-022	Edit a project (§2)	PROJ-009	update	EditProjectPage	(Should)	projects	PUT /projects/{id}	AC-PROJ-06
SRC-023	Delete a project (§2)	PROJ-010, §29	delete (cascade tx)	ConfirmDialog	(Should)	FK ON DELETE CASCADE	DELETE /projects/{id}	AC-PROJ-07
SRC-024	View all projects they own (§2)	PROJ-006	owner-scoped list	ProjectListPage	ProjectsScreen	projects(owner_id,...) idx	GET /projects	AC-PROJ-04

SRC	Assignment requirement (PDF section)	PRD IDs	Backend	Web	Mobile	Database	API	AC / TC
SRC-025	Project fields: Name, Description, Status, Start Date, End Date, Created Date (§2)	PROJ-001..005	schema + mapper	form/list	list/details	projects columns	Project shape	AC-PROJ-01..03
SRC-026	Status: Not Started, In Progress, Completed (§2)	PROJ-003	enum validation	badge/select	badge	enum project_status	status	AC-PROJ-03
SRC-030	Create tasks (§3)	TASK-001..006	tasks.service.create + project ownership	TaskForm	TaskForm screen	tasks	POST /tasks	AC-TASK-01..03
SRC-031	Edit tasks (§3)	TASK-011	update	TaskForm	TaskForm	tasks	PUT /tasks/{id}	AC-TASK-05
SRC-032	Delete tasks (§3)	TASK-014	delete	ConfirmDialog	ConfirmAlert	tasks	DELETE /tasks/{id}	AC-TASK-06
SRC-033	Mark tasks as completed (§3)	TASK-012, 013	idempotent status update	complete toggle	complete action	tasks.status	PUT /tasks/{id}	AC-TASK-07
SRC-034	View tasks under a project (§3)	TASK-009	projectId filter	ProjectDetails tasks	ProjectDetails tasks	tasks.project_id idx	GET /tasks?projectId	AC-TASK-09
SRC-035	Each project can contain multiple tasks (§3)	§16, TASK-003	FK check	—	—	tasks.project_id FK	POST /tasks	AC-TASK-01, AC-DB-01
SRC-036	Task fields: Name, Description, Priority, Status, Due Date, Created Date (§3)	TASK-001..006	schema + mapper	form/list	form/list	tasks columns	Task shape	AC-TASK-01..03
SRC-037	Priority: Low, Medium, High (§3)	TASK-004	enum	PriorityBadge	PriorityBadge	enum task_priority	priority	AC-TASK-03, 08
SRC-038	Status: Pending, In Progress, Completed (§3)	TASK-004	enum	StatusBadge	StatusBadge	enum task_status	status	AC-TASK-03, 08
SRC-040	Dashboard: Total Projects, Total Tasks, Completed Tasks, Pending Tasks, Projects In Progress (§4)	DASH-001, 002	dashboard.service COUNT/GROUP BY	StatCards	StatCards	aggregates	GET /dashboard	AC-DASH-01, 02
SRC-041	Dashboard reflects the authenticated user's data (§4)	DASH-004, 006	owner scoping	refetch on mutation	refetch + pull-to-refresh	—	GET /dashboard	AC-DASH-02, 04
SRC-050	Search projects by name (§5)	SRCH-001, 003	contains insensitive	SearchInput	(Should)	name search	GET /projects?search	AC-SRCH-01
SRC-051	Search tasks by name (§5)	SRCH-002	same	SearchInput	SearchInput	name search	GET /tasks?search	AC-SRCH-03
SRC-052	Filter projects by status (§5)	SRCH-001	status where	FilterBar	(Should)	idx (owner,status)	GET /projects?status	AC-SRCH-02

SRC	Assignment requirement (PDF section)	PRD IDs	Backend	Web	Mobile	Database	API	AC / TC
SRC-053	Filter tasks by status (§5)	SRCH-002	status where	FilterBar	FilterChips	idx (project,status)	GET /tasks? status	AC-SRCH-03
SRC-054	Filter tasks by priority (§5)	SRCH-002	priority where	FilterBar	FilterChips	idx (project,priority)	GET /tasks? priority	AC-SRCH-03
SRC-060	Mobile talks to the same backend/DB; no separate backend (§6)	MOB-002, SYNC-001	none added	—	single API client	—	all	AC-API-02, AC-SYNC-01
SRC-061	Register, log in, log out with the same account (§6)	MOB-003	auth endpoints	—	Login/Register/Profile	users	auth endpoints	AC-MOB-02
SRC-062	View the dashboard on mobile (§6)	MOB-004	dashboard	—	DashboardScreen	—	GET /dashboard	AC-MOB-03
SRC-063	View all projects and tasks under each (§6)	MOB-005	list + tasks filter	—	Projects, ProjectDetails	—	GET /projects, /tasks	AC-MOB-04
SRC-064	Create, edit, delete tasks on mobile (§6)	MOB-006	task CRUD	—	TaskForm, TaskDetails	tasks	POST/PUT/DELETE /tasks	AC-MOB-05
SRC-065	Mark completed; change status and priority on mobile (§6)	MOB-007	PUT	—	quick actions	tasks	PUT /tasks/{id}	AC-MOB-05, AC-TASK-08
SRC-066	Search tasks; filter by status and priority on mobile (§6)	MOB-008	query filters	—	Tasks screen	—	GET /tasks	AC-MOB-06
SRC-067	Android required; iOS optional (§6)	MOB-001	—	—	Expo Android build	—	—	AC-MOB-01
SRC-068	Changes appear on the other platform after refresh (pull-to-refresh) (§6)	SYNC-002, MOB-009	single source of truth	refetch/refresh	RefreshControl	—	GET endpoints	AC-MOB-07, AC-SYNC-01..03
SRC-069	Token stored in secure device storage (§6)	MOB-010	—	—	SecureStore	—	—	AC-MOB-08
SRC-070	Expired token → login screen with clear message (§6)	MOB-011, AUTH-017	401 TOKEN_EXPIRED	WEB-006 handler	API interceptor	—	any	AC-MOB-09, AC-AUTH-07
SRC-071	No network → clear message, no crash/blank (§6)	MOB-012	—	offline UI	offline UI	—	any	AC-MOB-10
SRC-080	Web: React or Next.js (Tech)	WEB-001	—	React + Vite	—	—	—	AC-DEP-02
SRC-081	Web: responsive design	UX-001	—	responsive layouts	—	—	—	AC-WEB-01
SRC-082	Web: proper component structure	WEB-002, 003	—	feature folders + ui/	—	—	—	AC-WEB-03
SRC-083	Web: form validation	WEB-008, UX-005	Zod (authoritative)	RHF + Zod	—	—	—	AC-WEB-02, AC-SEC-04

SRC	Assignment requirement (PDF section)	PRD IDs	Backend	Web	Mobile	Database	API	AC / TC
SRC-084	Web: loading indicators	UX-004, UX-009	—	skeleton/spinner	—	—	—	AC-WEB-02
SRC-085	Web: error handling	WEB-009, UX-011	error envelope	ErrorBoundary, <code>ErrorState</code>	—	—	—	AC-WEB-02, AC-API-03
SRC-086	Web: clean user experience	UX-001..018, UI-001..020	—	design tokens, states	—	—	—	AC-WEB-01, 02, AC-UI-01..06
SRC-090	Mobile: React Native (Expo or bare) or Flutter	MAPP-001	—	—	React Native + Expo	—	—	AC-MOB-01
SRC-091	Mobile: navigation and screen structure	MAPP-002, 003, MOB-014	—	—	React Navigation	—	—	AC-MAPP-01
SRC-092	Mobile: form validation	MOB-013, MAPP-008	Zod	—	RHF + Zod	—	—	AC-MAPP-01, AC-AUTH-03
SRC-093	Mobile: loading indicators and pull-to-refresh	MOB-009, MAPP-006, 009	—	—	RefreshControl, spinners	—	—	AC-MOB-07, AC-MAPP-01
SRC-094	Mobile: error handling incl. no network and expired login	MOB-011, 012	error envelope	—	interceptor + states	—	—	AC-MOB-09, 10
SRC-095	Mobile: secure token storage	MOB-010, SEC-013	—	—	<code>expo-secure-store</code>	—	—	AC-MOB-08
SRC-096	Mobile: clean UX on a phone screen	MOB-015, MAPP-010, UI-M01..M10	—	—	phone-first layouts	—	—	AC-MAPP-01
SRC-100	Backend: Express or NestJS; one backend for both apps	§15, MOB-002	Express modular monolith	—	—	—	—	AC-BE-01, AC-API-02
SRC-101	Backend: REST API architecture	§18, §19	REST resources + status codes	—	—	—	all	AC-API-01, 03
SRC-102	Backend: proper route organization	§15	<code>routes/*.routes.ts</code>	—	—	—	—	AC-BE-01
SRC-103	Backend: middleware usage	§15, SEC-004..018	auth, validate, limiter, helmet, cors	—	—	—	—	AC-BE-01, AC-SEC-02
SRC-104	Backend: error handling	§23, §18.1	<code>errorHandler</code>	—	—	—	all	AC-API-03, AC-SEC-12
SRC-105	Backend: logging	§15, NFR-009	pino + request id	—	—	—	—	AC-BE-01, AC-SEC-12
SRC-106	Backend: clean code structure	NFR-004, §15	layered modules	—	—	—	—	AC-BE-01
SRC-107	Backend: CORS configured for the web app's domain	SEC-010	<code>cors</code> allowlist	deployed origin	n/a	—	all	AC-SEC-08, AC-DEP-02
SRC-110	Database: PostgreSQL or MySQL	§16, DEP-004	Prisma + PostgreSQL	—	—	PostgreSQL	—	AC-DB-01, AC-DEP-04
SRC-111	Database: proper relational design	§16	schema	—	—	3 entities	—	AC-DB-01

SRC	Assignment requirement (PDF section)	PRD IDs	Backend	Web	Mobile	Database	API	AC / TC
SRC-112	Database: foreign key relationships	§16, DBS-003	FKs	—	—	projects.owner_id, tasks.project_id	—	AC-DB-01, AC-PROJ-07
SRC-113	Database: normalized structure	§16	no derived columns	—	—	3NF	—	AC-DB-01
SRC-120	Passwords hashed with bcrypt or equivalent (Security)	SEC-001, AUTH-006	bcrypt cost ≥ 12	—	—	password_hash	register	AC-AUTH-01, AC-SEC-01
SRC-121	Plain-text passwords never stored	SEC-002, DBS-001	redacted logs	—	—	no plaintext column	—	AC-SEC-01
SRC-122	Protected APIs require authentication	SEC-003, 004	authenticate	route guard	auth gate	—	all but public	AC-SEC-02
SRC-123	Users view/modify/delete only their own data, web and mobile	SEC-005, §21	ownership predicates	same API	same API	owner FK	all resource endpoints	AC-SEC-03
SRC-124	Users cannot access other users' data	§21	404 on foreign ids	—	—	—	all resource endpoints	AC-SEC-03, AC-PROJ-05, AC-TASK-04
SRC-125	Validate all incoming requests (required, email, dates, empty strings, enums)	SEC-006, PROJ-004/005, TASK-005	Zod strict schemas	client mirror	client mirror	CHECKs	write endpoints	AC-SEC-04, AC-AUTH-03, AC-PROJ-02/03, AC-TASK-03
SRC-126	JWT authentication	AUTH-011, §20	sign/verify HS256	stores token	stores token securely	—	auth endpoints	AC-AUTH-04
SRC-127	Authentication middleware	SEC-004	authenticate	—	—	—	protected	AC-SEC-02, AC-AUTH-07
SRC-128	Protected routes	SEC-004, WEB-005	deny-by-default mounting	Protected Route	auth gating	—	protected	AC-SEC-02, AC-AUTH-08
SRC-129	Sensitive information not exposed in responses	SEC-007, AUTH-015	DTO mappers	—	—	—	all	AC-SEC-05
SRC-130	SQL-injection protection (ORM/parameterization)	SEC-008, DBS-002	Prisma only	—	—	—	all	AC-SEC-06, AC-SRCH-05
SRC-131	Rate limiting on authentication endpoints	SEC-009, AUTH-013	express-rate-limit	429 message	429 message	—	login, register	AC-AUTH-05, AC-SEC-07
SRC-140	Auth endpoints: POST /api/auth/register, POST /api/auth/login, POST /api/auth/logout, GET /api/auth/me	API-01..04	auth routes	auth API module	auth API module	users	those four	AC-API-01, AC-AUTH-01..08

SRC	Assignment requirement (PDF section)	PRD IDs	Backend	Web	Mobile	Database	API	AC / TC
SRC-141	Project endpoints: GET /api/projects, GET /api/projects/{id}, POST /api/projects, PUT /api/projects/{id}, DELETE /api/projects/{id}	API-05..09	project routes	projects API	projects API	projects	those five	AC-API-01, AC-PROJ-01..07
SRC-142	Task endpoints: GET /api/tasks, GET /api/tasks/{id}, POST /api/tasks, PUT /api/tasks/{id}, DELETE /api/tasks/{id}	API-10..14	task routes	tasks API	tasks API	tasks	those five	AC-API-01, AC-TASK-01..07
SRC-143	Dashboard endpoint: GET /api/dashboard	API-15	dashboard route	dashboard API	dashboard API	aggregates	that one	AC-API-01, AC-DASH-01..04
SRC-144	Web and mobile must both use these endpoints	API-001, AC-API-02	—	uses them	uses them	—	all 15	AC-API-02
SRC-160	Docs: project setup (backend, web, mobile)	DOC-001	README	README	README	—	—	AC-DOC-01
SRC-161	Docs: environment variable documentation	DOC-002, §32	.env.example	.env.example	.env.example	—	—	AC-DOC-01, AC-SEC-09
SRC-162	Docs: database setup instructions	DOC-003	migrate/seed docs	—	—	migrations	—	AC-DOC-01
SRC-163	Docs: API documentation	DOC-004	openapi.yaml, /api/docs	—	—	—	all	AC-DOC-02
SRC-164	Docs: run mobile against deployed backend	DOC-005, DEP-003	—	—	env + APK steps	—	—	AC-DOC-01, AC-DEP-03
SRC-165	Easy to run by another developer	DOC-007, NFR-010	scripts	scripts	scripts	seed	—	AC-DOC-01
SRC-170	Submit: public GitHub repository link	GIT-001, SUB-001	—	—	—	—	—	AC-SUB-01
SRC-171	Submit: database schema or ER diagram	DOC-006, SUB-002	—	—	—	schema docs	—	AC-DOC-03
SRC-172	Submit: API documentation	SUB-003	—	—	—	—	—	AC-DOC-02
SRC-173	Submit: README file	SUB-004	—	—	—	—	—	AC-DOC-01
SRC-174	Submit: deployment URL for web app and backend	DEP-001, 002, SUB-005	deployed	deployed	—	managed DB	—	AC-DEP-01, 02

SRC	Assignment requirement (PDF section)	PRD IDs	Backend	Web	Mobile	Database	API	AC / TC
SRC-175	Submit: Android APK or Expo/Firebase distribution link	DEP-003, SUB-006	—	—	EAS APK	—	—	AC-DEP-03
SRC-176	Submit: 5-minute screen recording (same account on web and mobile; task created on one, shown on the other)	§35, SUB-007	—	demo	demo	—	—	AC-SUB-02
SRC-180	Bonus: Docker, Unit Tests, Integration Tests, CI/CD Pipeline	§37 (B)	docker-compose, tests, workflow	tests	tests	—	—	own checks if built
SRC-181	Bonus: Pagination, Sorting, Refresh Tokens, Role-Based Access Control, Audit Logs	§37 (B)	optional modules	optional UI	optional UI	optional tables	additive params	own checks if built
SRC-182	Bonus: Push Notifications for tasks due tomorrow, Offline viewing of tasks on mobile, Shared types or validation	§37 (B)	job / shared package	shared package	notification s, cache	optional	—	own checks if built
SRC-200	Libraries, frameworks, AI-assisted tools and online resources may be used (Notes)	§43	—	—	—	—	—	n/a
SRC-201	Candidate must be able to explain implementation and design decisions (Notes)	§42 ADRs, README decisions	—	—	—	—	—	Review session
SRC-202	Readability, maintainability and security practices are evaluation criteria (Notes)	NFR-004, §22	layered code	structured code	structured code	—	—	AC-BE-01, AC-WEB-03, AC-SEC-*
SRC-203	Not required to be production-ready but should show good engineering practice (Notes)	§5, §48 principles	—	—	—	—	—	§45 gate
SRC-204	Use only test data; no real personal data (Notes)	SUB-008, DBS-007	synthetic seed	—	—	synthetic rows	—	AC-SUB-01 checklist (SUB-008)

39. Definition of Done

A feature is NOT done because its UI exists. A feature is **done** only when ALL of the following hold:

1. UI exists on every platform where it is required (web; Android where required).
2. The API endpoint exists, matches §18, and is documented in OpenAPI.
3. Backend validation exists and rejects invalid input with 400 and field details.
4. Authorization and ownership checks exist and are covered by a cross-user check.
5. Database behavior works (migrations applied, constraints and indexes in place, cascade rules verified).
6. Error handling exists (standard envelope; friendly client messages).
7. Loading state exists.
8. Empty state exists.
9. Tests exist where applicable (at minimum the P0 tests for that feature) and pass.
10. Web behavior works end to end against the real backend.
11. Mobile behavior works end to end where applicable, on an Android device or emulator.
12. Documentation is updated (README, OpenAPI, env table, ER diagram if schema changed).
13. Security requirements of §22 are satisfied for the feature (no secrets, no sensitive data in responses, no raw SQL).
14. Its acceptance criteria (AC-*) pass and the traceability row is ticked.

40. Implementation Phasing

Phase	Objectives	Deliverables	Dependencies	Exit criteria	Risks
0 Setup and architecture	Repo, tooling, conventions	Monorepo skeleton, TypeScript/ESLint/Prettier, <code>.gitignore</code> , <code>.env.example</code> s, README stub, ADR notes	None	All three apps boot a hello screen/route; lint and type-check pass	Tool/SDK version mismatches (Node, Expo)
1 Database and backend foundation	Schema and API skeleton	Prisma schema + migrations (CHECKs), <code>config.ts</code> , <code>app.ts</code> , logging, helmet, CORS, error handler, <code>/api/health</code> , test harness	0	Migrations apply to an empty DB; health 200; error-envelope tests pass; AC-DB-01	Migration issues, enum/CHECK mistakes
2 Authentication	Secure accounts	Register/login/logout/me, bcrypt, JWT, <code>authenticate</code> , auth rate limit	1	AC-AUTH-01..08, AC-SEC-02 pass	Auth bugs, JWT misconfig, wrong client IP behind proxy

Phase	Objectives	Deliverables	Dependencies	Exit criteria	Risks
3 Project management	Projects CRUD	Project routes/services with ownership, counts, cascade delete	2	AC-PROJ-01..07, 09 pass	IDOR/ownership gaps
4 Task management	Tasks CRUD	Task routes/services with project-based ownership, quick status/priority updates	3	AC-TASK-01..07, 09 pass	Foreign <code>projectId</code> bugs, partial-update edge cases
5 Dashboard, search, filtering	Stats and querying	<code>/api/dashboard</code> , <code>search/status/priority/projectId</code> filters, OpenAPI draft	3, 4	AC-DASH-01, 02; AC-SRCH-01..05 pass	Count mismatches; wildcard escaping
6 Web frontend	Complete web app	Auth screens, protected routes, dashboard, projects, tasks, search/filter, states, responsive layout	2–5	AC-WEB-01..03, AC-PROJ-08, AC-TASK-08..10, AC-DASH-03/04, AC-SRCH-06 pass	UI scope creep; CORS
7 Mobile frontend	Complete Android app	Navigation, auth with SecureStore, dashboard, projects, tasks, search/filter, pull-to-refresh, offline/expiry handling	2–5	AC-MOB-01..10, AC-MAPP-01 pass on a device	Emulator networking, cleartext/HTTPS, Expo build issues
8 Cross-platform sync	Prove shared data	S1–S4 scenarios; cache invalidation tuned	6, 7	AC-SYNC-01..03, AC-AUTH-09 pass	Stale caches, token confusion between environments
9 Security hardening	Verify and fix	Full AC-SEC set, headers, secret scan, injection tests, rate-limit tests	2–8	AC-SEC-01..12 pass; no secrets in history	Late-found authorization gaps
10 Testing	Raise confidence	Complete P0 suite, unit tests, edge cases of §28, defect fixes	2–9	All P0 tests green; no open Must defects	Flaky tests, DB state leakage
11 Deployment	Go live	Managed DB, backend, web, APK; <code>prisma migrate deploy</code> ; CORS/env set; smoke test	10	AC-DEP-01..04 pass	Env misconfig, CORS, cold starts, APK pointing to wrong URL
12 Documentation and final demo	Package the submission	README, OpenAPI, ER diagram, demo rehearsal + recording, submission checklist, final §45 gate	11	AC-DOC-01..03, AC-SUB-01..02 pass; gate fully ticked	Last-minute breakage, unclear README

40.1 Core-First Delivery: Vertical Slices

The core required functionality is built **end-to-end, slice by slice**, instead of finishing all backend work before any UI. Phases 2–8 above are executed through the slices below. A slice is done only when it works on **web and Android against the real backend**, with UI

built from the design system of §12.2 at the quality bar of §12.6 (no “ugly first, polish later”). This section is a **RECOMMENDED ENGINEERING DECISION**; it reorders work but removes no requirement.

Slice	End-to-end scope	Exit criteria	Check on both platforms
0 Foundation + design system	Repo, PostgreSQL schema + migrations, config, middleware, <code>/api/health</code> ; web: Tailwind + shadcn/ui init, tokens, AppShell, base components (Button, Input, Badge, Dialog, Toast, Skeleton, EmptyState, ErrorState); mobile: Expo + Paper theme from the same tokens, navigation shell, API client, SecureStore wrapper	AC-DB-01; tokens file exists; shells render	Both apps boot against a local backend
1 Authentication	Register/login/logout/me, bcrypt, JWT, rate limit; web Login/Register, <code>ProtectedRoute</code> , logout; mobile Splash/Login/Register/Profile, SecureStore, expired-token flow	AC-AUTH-01..09, AC-SEC-02, AC-MOB-02, AC-MOB-08, AC-MOB-09	Same account logs in on web and Android
2 Projects	Project CRUD + ownership + counts; web list/details/create/edit/delete with confirmation; mobile list + details	AC-PROJ-01..09	Project created on web visible on Android after refresh
3 Tasks	Task CRUD + project-based ownership; web project tasks + global Tasks + quick complete/status/priority; mobile task CRUD + quick actions	AC-TASK-01..10, AC-MOB-04, AC-MOB-05	Task created on one platform shown on the other (demo scenario)
4 Dashboard	<code>/api/dashboard</code> ; web StatCards (+ status bar); mobile stat grid	AC-DASH-01..04, AC-MOB-03	Same five numbers on both platforms

Slice	End-to-end scope	Exit criteria	Check on both platforms
5 Search and filtering	search / status / priority / projectId params; web toolbars; mobile Searchbar + chips	AC-SRCH-01..06, AC-MOB-06	Same filter gives same results on both
6 Sync, resilience, hardening	Pull-to-refresh everywhere, offline and expiry handling on every screen, security and injection tests, UI quality-bar pass	AC-SYNC-01..03, AC-MOB-07, AC-MOB-10, AC-SEC-01..12, AC-UI-01..06	S1-S4 scenarios pass
7 Deploy, docs, demo	Managed DB, backend, web, APK; README, OpenAPI, ER diagram; demo recording	AC-DEP-01..04, AC-DOC-01..03, AC-SUB-01..02, §45 gate	Demo script (§35) runs on deployed apps

Rules: (1) a slice starts only after the previous slice's exit criteria pass; (2) the slice's OpenAPI contract is written before its UI; (3) every slice ends with a demo on both platforms and a Git tag `slice-N-done`; (4) deploy the backend, database and web app after Slice 1 and produce the first APK after Slice 2-3, so deployment and build risks surface early; (5) optional features (§37, UI-P1 items) wait until Slice 7 is complete.

41. Risk Register

Risk	Probability	Impact	Mitigation
Authentication bugs (token handling, hashing, validation)	Medium	High	Build auth first (Phase 2); AC-AUTH tests; pinned JWT algorithm; code review of <code>authenticate</code>
Authorization / data leakage (IDOR/BOLA)	Medium	Critical	Owner predicate in the data layer; two-user matrix test (AC-SEC-03) for every resource endpoint; uniform 404s

Risk	Probability	Impact	Mitigation
Mobile ↔ backend connectivity (emulator localhost, HTTPS, firewall)	High	High	Document <code>10.0.2.2</code> /LAN setup; test early against the deployed HTTPS backend; clear offline UI
CORS misconfiguration blocks the web app	High	High	Exact-origin allowlist in <code>CORS_ORIGIN</code> ; browser test from the deployed origin (AC-SEC-08, AC-DEP-02); document in README
JWT expiration handled inconsistently	Medium	Medium	One shared 401 interceptor per client; short-lived-token manual test (AC-MOB-09); one message string
Database migration issues (drift, failed deploy)	Medium	High	Prisma Migrate only; test on a clean DB; run <code>migrate deploy</code> in the release step; backups
Deployment configuration errors (env vars, wrong API URL baked into APK)	High	High	Env table (§32); smoke-test checklist; verify APK against the deployed URL on a clean device
API inconsistency between web and mobile	Medium	Medium	One OpenAPI contract; identical enums/shapes; optional shared schemas (bonus); AC-API-02 grep
Network failures / timeouts / free-tier cold starts	High	Medium	15 s timeout, offline UI with Retry; warm the backend before demo (DEP-005)
Scope creep	Medium	High	Non-goals (§5); P0/P1/P2 rule (§44); changes recorded in §47
Bonus features delaying mandatory work	Medium	High	Bonus only after the §45 gate; strict ordering in §37
Secrets committed to the public repo	Low	Critical	<code>.gitignore</code> , <code>.env.example</code> , secret scan (SEC-022), history review before publishing

Risk	Probability	Impact	Mitigation
Expo/EAS build failures close to the deadline	Medium	High	Produce a first APK in Phase 7 (not at the end); keep dependencies minimal
Rate limiter blocks evaluators behind a shared IP	Low	Low	Count only failed logins; documented limits; env-configurable

42. Architecture Decision Records

ADR	Decision	Reason	Alternatives	Trade-offs
ADR-001	React + Vite + TypeScript SPA for web	PDF allows React or Next.js; an authenticated app on a REST API needs no SSR; fast, simple, widely understood	Next.js	No SSR/SEO (irrelevant here); static hosting needs SPA fallback
ADR-002	React Native + Expo for Android	Same language/skills as web; EAS Build gives an APK quickly; SecureStore is built in	Flutter; bare React Native	Managed workflow limits some native modules; Expo Go is not enough for every feature
ADR-003	Node.js + Express (TypeScript) backend	Minimal, explainable, flexible for a modular monolith	NestJS	Structure is not enforced by the framework, so the layering in §15 is imposed by convention
ADR-004	PostgreSQL	Strong relational integrity, enums, FKs, CHECKs; good managed free tiers	MySQL	Requires a managed/hosted instance for deployment
ADR-005	Prisma ORM	Parameterized queries by default, typed client, migrations	TypeORM, Knex, raw pg	CHECK constraints need hand-written SQL migrations; extra engine at build time
ADR-006	JWT in the Authorization header (stateless)	One mechanism for web and mobile; matches PDF; no cookies, so no CSRF surface	Server sessions; cookies	No revocation (OD-03); a stolen token works until expiry

ADR	Decision	Reason	Alternatives	Trade-offs
ADR-007	bcrypt (<code>bcryptjs</code> acceptable), cost 12	PDF names bcrypt; mature and simple	argon2, scrypt	72-byte input limit (enforced); pure-JS variant is slower than native
ADR-008	Ownership-based authorization resolved through projects	Matches a single-owner data model; simple and verifiable	RBAC, per-row ACLs	No sharing/collaboration (a non-goal)
ADR-009	One backend, one database, modular monolith	PDF requires a single backend for both clients; least complexity	Separate mobile backend; microservices	One deployable (acceptable at this scale)
ADR-010	Expo SecureStore (Android Keystore) for the mobile token	PDF requires secure device storage	AsyncStorage (rejected), <code>react-native-keychain</code>	Tied to Expo modules; value size limits are fine for a JWT
ADR-011	Foreign/missing resources return 404, not 403	Avoids revealing that another user's id exists	403	Slightly less specific error for legitimate typos (OD-11)
ADR-012	Cascade-delete tasks when a project is deleted, with explicit confirmation	Clear, atomic, no orphan or blocked states	Restrict; soft delete	Irreversible; mitigated by confirmation (OD-02)
ADR-013	TanStack Query for server state + React Context for auth/UI	Handles caching, refetch, invalidation and pull-to-refresh with little code	Redux, ad-hoc <code>useEffect</code>	One more dependency
ADR-014	Zod strict schemas for validation; clients mirror rules	Typed, composable, rejects unknown fields (mass-assignment defense)	Joi, class-validator	Duplicated rules unless shared (bonus)
ADR-015	<code>PUT</code> accepts partial updates	Enables quick complete/status/priority changes through the mandated endpoints	Full-replace PUT; extra PATCH endpoints	Not strictly REST-pure (OD-07)
ADR-016	Web token in <code>localStorage</code>	Persistent login with a stateless API and no cookie/CORS-credential complexity	httpOnly cookie	XSS exposure; mitigated by CSP and no raw HTML (OD-06)

43. ANTIGRAVITY Implementation Instructions

Antigravity is the primary implementer. It MUST:

1. Read this PRD completely before implementation.
2. Treat every **M** requirement as non-negotiable.
3. Never remove a requirement to simplify implementation.
4. Never expose secrets (no secrets in code, logs, client bundles, or Git).
5. Never bypass backend authorization.
6. Never trust frontend validation alone.
7. Never create a second mobile backend or an independent mobile database for primary data.
8. Keep web and mobile API contracts identical (same paths, payloads, enums, error envelope).
9. Keep the database relational and normalized.
10. Use migrations for every schema change.
11. Keep code modular (routes → middleware → controllers → services → data access).
12. Write tests alongside important functionality (P0 tests first).
13. Validate APIs independently (curl/Supertest) before building UI on top.
14. Handle loading, empty and error states on every screen.
15. Implement security before polishing optional features.
16. Maintain documentation as implementation evolves.
17. Avoid unnecessary dependencies.
18. Avoid overengineering; prefer a simple, maintainable architecture the candidate can explain.
19. Prefer the PDF over this PRD wherever they appear to conflict, and record the conflict in §47 instead of guessing.
20. Before declaring completion, run a requirement-by-requirement verification against the §38 traceability matrix and the §45 gate.

Additional operating rules (R): follow the phase order in §40 executed as the core-first vertical slices of §40.1; build every screen with the design system and quality bar of §12.1–§12.6 from the first slice; reference requirement IDs in commit messages and tests; never merge a feature that fails its AC; use synthetic data only; when something is ambiguous, choose the safest option, document it in §47, and continue.

44. Implementation Order Rule

Antigravity **MUST** prioritize:

- **P0 (mandatory, in this order):** core architecture → database → authentication → authorization → projects → tasks → dashboard → search/filtering → required APIs → web functionality → Android functionality → cross-platform synchronization → security → error handling → deployment → documentation, with the design system and UI/UX quality bar of §12.1–§12.6 applied to every core screen from the first slice (§40.1).
- **P1:** extras beyond the core quality bar (dark theme, board view, dashboard status chart, Lighthouse tuning), testing improvements, performance improvements, additional engineering quality (e.g. mobile project create/edit/delete, project search on mobile).
- **P2:** bonus features (§37).

Never implement P2 while any P0 requirement is incomplete. Never start P1 polish while a P0 acceptance criterion is failing.

45. Final Quality Gate

Before the project is complete, verify every item:

- ☐ All mandatory assignment requirements implemented (traceability matrix fully ticked)
- ☐ All required API endpoints implemented (15 of 15)
- ☐ Web application functional
- ☐ Android application functional
- ☐ Same account works on both
- ☐ Shared backend
- ☐ Shared database
- ☐ Projects CRUD works
- ☐ Tasks CRUD works
- ☐ Dashboard works
- ☐ Search works
- ☐ Filtering works
- ☐ JWT authentication works
- ☐ bcrypt password hashing works
- ☐ Authorization prevents cross-user access
- ☐ Backend validation works
- ☐ SQL injection protection implemented
- ☐ Authentication rate limiting implemented
- ☐ Secure mobile token storage implemented
- ☐ Token expiration handled

- ☐ No-network handling implemented
- ☐ Loading states implemented
- ☐ Error states implemented
- ☐ Empty states implemented
- ☐ CORS configured
- ☐ Database schema documented
- ☐ API documented
- ☐ README complete
- ☐ Environment variables documented
- ☐ Backend deployed
- ☐ Web deployed
- ☐ Android build/distribution available
- ☐ 5-minute demo prepared
- ☐ No secrets committed
- ☐ Tests passing
- ☐ Traceability matrix fully satisfied

Added in v1.1:

- ☐ Core slices 0–6 completed in order (§40.1), each demonstrated on web and Android against the real backend
- ☐ UI quality bar of §12.6 met on web and Android (AC-UI-01..05 pass); design tokens shared by both apps
- ☐ Open-source UI sources credited in `THIRD_PARTY_NOTICES.md` and the README

46. Requirement Coverage Audit

Result: every mandatory requirement in the source PDF is represented in this PRD, with an ID, an owner section, and at least one acceptance criterion. The PDF yields 111 source requirement rows (SRC-001 to SRC-204) in §38.2; none is omitted, none was silently downgraded from mandatory, and none of the 12 bonus items was promoted to mandatory.

PDF section	SRC rows	Where specified	Status
Introduction	SRC-001..004	§2, §7, §8, §15	Covered

PDF section	SRC rows	Where specified	Status
1 User Authentication (registration, login, logout, fields, unique email, no plaintext, stay logged in, one account)	SRC-010..017	§8A, §20, §22	Covered
2 Project Management (create, view, edit, delete, list owned, 6 fields, 3 statuses)	SRC-020..026	§8B, §16, §29	Covered
3 Task Management (create, edit, delete, complete, view under project, 6 fields, 3 priorities, 3 statuses)	SRC-030..038	§8C, §16	Covered
4 Dashboard (5 metrics, per-user)	SRC-040..041	§8D, §18.5	Covered
5 Search and Filtering (5 capabilities)	SRC-050..054	§8E, §30	Covered
6 Mobile App (same backend, same account, dashboard, projects/tasks, task CRUD, complete/status/priority, search/filter, Android/iOS, refresh, secure storage, expiry, no network)	SRC-060..071	§8F, §8G, §14	Covered
Technical: Frontend (Web)	SRC-080..086	§12, §13	Covered
Technical: Mobile App	SRC-090..096	§14	Covered
Technical: Backend	SRC-100..107	§15, §23	Covered
Technical: Database	SRC-110..113	§16, §17	Covered
Security Requirements	SRC-120..131	§20–§22	Covered
API Expectations (15 endpoints, used by both apps)	SRC-140..144	§18, §19	Covered
Documentation Requirements	SRC-160..165	§33, §32	Covered
Submission Requirements (7 items)	SRC-170..176	§35, §36	Covered

PDF section	SRC rows	Where specified	Status
Bonus Features (12 items)	SRC-180..182	§37 (all labeled B)	Covered as OPTIONAL
Important Notes	SRC-200..204	§42, §43, §36	Covered

The PDF's alternatives were resolved as: React (not Next.js), React Native + Expo (not Flutter), Express (not NestJS), PostgreSQL (not MySQL). Items this PRD adds beyond the PDF (OpenAPI format, health endpoint, password policy, field limits, rate-limit values, test tooling, deployment providers, `taskCount` fields, `inProgressTasks`) are labeled R.

47. Open Decisions / Clarifications

The PDF is silent or ambiguous on the points below. Each has a recommended decision applied in this PRD; none was assumed silently. Change a decision only by updating the cited requirement IDs and acceptance criteria.

ID	Topic	PDF position	Decision applied (R)	Affects
OD-01	Are Start Date and End Date required?	Lists them as fields; no required/optional statement	Optional and nullable; if both set, End ≥ Start	PROJ-004, AC-PROJ-03
OD-02	Deleting a project that has tasks	Not specified	Cascade delete with confirmation showing task count (§29)	PROJ-010, AC-PROJ-07
OD-03	Server-side behavior of logout	Requires the endpoint only; JWT-based	Stateless, idempotent 204; clients clear tokens; no revocation	AUTH-014, AC-AUTH-08
OD-04	Meaning of Pending Tasks	Task status <code>Pending</code> exists; metric name "Pending Tasks"	Count of tasks with status <code>PENDING</code> ; extra <code>inProgressTasks</code> so totals reconcile	DASH-002, 003
OD-05	Token lifetime	"until logout or token expiration"	<code>JWT_EXPIRES_IN</code> default <code>1d</code> , configurable	AUTH-012

ID	Topic	PDF position	Decision applied (R)	Affects
OD-06	Web token storage	Secure storage mandated for mobile only	localStorage plus CSP and no raw HTML; httpOnly cookie is the alternative	WEB-010
OD-07	PUT semantics	Endpoint listed as PUT	Partial update with at least one field	PROJ-009, TASK-011
OD-08	Linking tasks to projects in the API	Only /api/tasks endpoints listed	projectId in create body and list query; immutable on update	TASK-001, 009, 011
OD-09	Does registration sign the user in?	"Register" and "log in" only	Yes: returns user + token	AUTH-008
OD-10	Validation status code	"Appropriate error responses"	400 for all validation failures (422 unused)	§19
OD-11	Status for another user's resource	"Must not be able to access"	404 (no existence leak); 403 reserved	PROJ-011, §21
OD-12	Project create/edit/delete and project search on mobile	Mobile list requires viewing projects and task CRUD only	Should (R), after mandatory mobile items	MOB-016, PROJ-015
OD-13	Field length limits and password policy	Silent	Name 120 / task 150 / description 2000 / full name 100; password 8-72 chars with a letter and a digit	AUTH-002, 005, PROJ-001, TASK-001
OD-14	Is Created Date client-settable?	"Created Date" listed	Server-set only	PROJ-002, TASK-002
OD-15	Past due dates	Silent	Allowed; shown as Overdue until completed	TASK-005, 016
OD-16	List size / pagination	Pagination is bonus	Lists return all matching rows in v1; pagination is additive later	§18, §37

ID	Topic	PDF position	Decision applied (R)	Affects
OD-17	Android minimum version and rate-limit values	Silent	Android 8.0 (API 26)+; login 10 failed/15 min, register 10/hour	MOB-001 context, AUTH-013